

B_Tech_M_Tech__Report_Format_Latex__1__3_(no rfe).pdf

 SAKEC - Shah and Anchor Kutchhi Engineering College

Document Details

Submission ID

trn:oid::3618:137316744

Submission Date

May 1, 2026, 1:45 PM GMT+5:30

Download Date

May 1, 2026, 1:47 PM GMT+5:30

File Name

B_Tech_M_Tech__Report_Format_Latex__1__3_(no rfe).pdf

File Size

1.6 MB

88 Pages

18,493 Words

109,283 Characters

8% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **122 Not Cited or Quoted 8%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **1 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 5%  Internet sources
- 2%  Publications
- 6%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 122 Not Cited or Quoted 8%**
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%**
Matches that are still very similar to source material
- 1 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 5% Internet sources
- 2% Publications
- 6% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	www.coursehero.com	2%
2	Submitted works	Charles University in Prague on 2026-04-29	<1%
3	Submitted works	Alliance University on 2026-04-13	<1%
4	Internet	arxiv.org	<1%
5	Submitted works	University of Mauritius on 2026-04-06	<1%
6	Submitted works	Nottingham Trent University on 2026-04-30	<1%
7	Internet	github-wiki-see.page	<1%
8	Submitted works	Middlesex University on 2026-04-12	<1%
9	Submitted works	City University of Hong Kong on 2025-10-28	<1%
10	Submitted works	University of Hertfordshire on 2026-04-03	<1%

11	Internet	claude-plugins.dev	<1%
12	Submitted works	UCL on 2025-09-08	<1%
13	Submitted works	Erasmus University of Rotterdam on 2025-04-20	<1%
14	Submitted works	Leeds Beckett University on 2026-04-22	<1%
15	Internet	pdfcoffee.com	<1%
16	Submitted works	Flinders University on 2025-05-31	<1%
17	Submitted works	Harrisburg University of Science and Technology on 2025-10-06	<1%
18	Submitted works	IUBH - Internationale Hochschule Bad Honnef-Bonn on 2026-04-24	<1%
19	Submitted works	Manipal Academy of Higher Education (MAHE) on 2026-04-30	<1%
20	Submitted works	University of Wales Institute, Cardiff on 2026-03-10	<1%
21	Submitted works	University of Leicester LTI on 2025-05-02	<1%
22	Submitted works	University of Wolverhampton on 2026-04-29	<1%
23	Internet	his.diva-portal.org	<1%
24	Internet	mobile.wiredspace.wits.ac.za	<1%

25	Submitted works	Bindura University of Science Education on 2026-04-20	<1%
26	Submitted works	Dublin City University on 2026-04-30	<1%
27	Submitted works	Liverpool John Moores University on 2010-01-23	<1%
28	Submitted works	University of Wales Institute, Cardiff on 2026-03-05	<1%
29	Submitted works	APJ Abdul Kalam Technological University, Thiruvananthapuram on 2025-03-23	<1%
30	Submitted works	S.P. Jain Institute of Management and Research, Mumbai on 2026-02-11	<1%
31	Internet	glama.ai	<1%
32	Internet	seejph.com	<1%
33	Internet	theses.lib.polyu.edu.hk	<1%
34	Submitted works	Liverpool John Moores University on 2024-07-28	<1%
35	Publication	Prechel, Garrett Lyndon. "Integrated Fast-Ion Diagnostics: Hardware Developme..."	<1%
36	Submitted works	Strathmore University (Main Account) on 2025-11-28	<1%
37	Submitted works	University of Sunderland on 2025-11-04	<1%
38	Internet	patentimages.storage.googleapis.com	<1%

39	Internet	repository.enp.edu.dz	<1%
40	Submitted works	African Leadership University on 2026-03-22	<1%
41	Submitted works	Ton Duc Thang University on 2026-01-05	<1%
42	Submitted works	University of Edinburgh on 2023-04-25	<1%
43	Internet	eprints.utar.edu.my	<1%
44	Internet	www.termpaperwarehouse.com	<1%
45	Submitted works	Asian Institute of Technology on 2012-05-20	<1%
46	Submitted works	Berner Fachhochschule on 2026-01-04	<1%
47	Submitted works	Islington College,Nepal on 2026-04-21	<1%
48	Submitted works	UCL on 2024-09-09	<1%
49	Submitted works	Universitat Oberta de Catalunya on 2026-03-07	<1%
50	Internet	etheses.whiterose.ac.uk	<1%
51	Internet	huggingface.co	<1%
52	Internet	www.ifad.org	<1%

53	Submitted works	Anglia Ruskin University on 2013-05-17	<1%
54	Submitted works	Kingston University on 2025-09-18	<1%
55	Submitted works	University of Malaya on 2025-09-10	<1%
56	Internet	agilebrandguide.com	<1%
57	Internet	ap-st01.ext.exlibrisgroup.com	<1%
58	Internet	publications.rwth-aachen.de	<1%
59	Submitted works	Crown Institute of Business and Technology on 2025-10-12	<1%
60	Submitted works	Islington College,Nepal on 2026-04-22	<1%
61	Submitted works	Kingston University on 2025-08-20	<1%
62	Submitted works	SSBAS on 2026-04-18	<1%
63	Submitted works	University of Stirling on 2025-04-08	<1%
64	Submitted works	University of Westminster on 2026-04-17	<1%
65	Internet	ir.library.osaka-u.ac.jp	<1%
66	Internet	ladal.edu.au	<1%

67	Internet	pure.tue.nl	<1%
68	Internet	pypi.org	<1%
69	Internet	www.frontiersin.org	<1%
70	Submitted works	Islington College,Nepal on 2026-04-22	<1%
71	Submitted works	Islington College,Nepal on 2026-04-22	<1%
72	Publication	Jakkaphong Inpun, Watcharaporn Chalamjiak, Piyada Phrueksawatnon, Kanokwa...	<1%
73	Submitted works	Lahore Garrison University-Lahore. on 2025-11-14	<1%
74	Submitted works	Middlesex University on 2025-07-08	<1%
75	Submitted works	SSBAS on 2026-04-13	<1%
76	Submitted works	Sir Syed University of Engineering and Technology, Karachi on 2026-04-19	<1%
77	Submitted works	Universidade do Porto on 2024-06-30	<1%
78	Submitted works	University of Bedfordshire on 2026-04-25	<1%
79	Submitted works	University of Greenwich on 2026-04-15	<1%
80	Submitted works	University of Greenwich on 2026-04-22	<1%

81	Publication	Vincent G. Duffy. "Advances in Applied Human Modeling and Simulation", CRC Pr...	<1%
82	Internet	aukro.sk	<1%
83	Internet	codeberg.org	<1%
84	Internet	etd.aau.edu.et	<1%
85	Internet	theses.hal.science	<1%
86	Internet	www.se.cs.uni-saarland.de	<1%
87	Submitted works	South Bank University on 2026-03-20	<1%
88	Submitted works	University of Wales Institute, Cardiff on 2026-02-26	<1%
89	Publication	Jaskaran Singh, Meenu Gupta, Rakesh Kumar. "Smart Technologies and Intelligen...	<1%

Engunity X : AI-Powered SaaS Platform for Research, CodeGeneration and Data Analysis

A

DISSERTATION/REPORT

Submitted for fulfillment of the Requirement

For the award of the degree of

BACHELOR OF TECHNOLOGY

In

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Submitted by

Sr. No.	Name	Roll No.	Email ID
1	Shubh Shah	22UF17191AI053	shubh.17191@sakec.ac.in
2	Anurag Kumar	22UF17647AI027	anurag.kumar17647@sakec.ac.in
3	Shreyash Mokani	22UF17269AI032	shreyash.17269@sakec.ac.in
4	Nigranth Shah	22UF17430AI051	nigranth.17430@sakec.ac.in

Under the guidance of

Dr. Rahul Pachade

Associate Professor



**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA
SCIENCE**

SHAH & ANCHOR KUTCHHI ENGINEERING COLLEGE

(An Autonomous Institute Affiliated to University of Mumbai)

MUMBAI - 400 088, MAHARASHTRA (INDIA)

MAY 2026



Shah & Anchor Kutchhi Engineering College
(An Autonomous Institute Affiliated to University of Mumbai)
Mumbai-400 088, MAHARASHTRA (India)

CERTIFICATE

It is my pleasure to certify that worked under my supervision for the B. Tech. dissertation/ Project report entitled Engunity X : AI-Powered SaaS Platform for

Sr. No.	Name	Roll No.	Email ID
1	Shubh Shah	22UF17191AI053	shubh.17191@sakec.ac.in
2	Anurag Kumar	22UF17647AI027	anurag.kumar17647@sakec.ac.in
3	Shreyash Mokani	22UF17269AI032	shreyash.17269@sakec.ac.in
4	Nigranth Shah	22UF17430AI051	nigranth.17430@sakec.ac.in

Research, Code Generation and Data Analysis and his/her work is of the level of requirement set up for the dissertation/Project Report in ARTIFICIAL INTELLIGENCE AND DATA SCIENCE by Shah & Anchor Kutchhi Engineering College, Mumbai.

Date: 16/05/2026

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Dr. Rahul Pachade

Associate Professor

Department of Artificial Intelligence and Data Science

Shah & Anchor Kutchhi Engineering College

Mumbai – 400 088, India

The viva-voice/Oral and Practical examination of , B. Tech. in ARTIFICIAL INTELLIGENCE AND DATA SCIENCE, has been held on 16/05/2026

External Examiner

Internal Examiner

Guide Name

(Dr. Rahul Pachade)

I/C Head of Department

(Dr. Pravin Shinde)

Principal

(Dr. Bhavesh Patel)

College Seal

CANDIDATE'S DECLARATION

We hereby declare that the work presented in the dissertation/Project Report entitled Engunity X : AI-Powered SaaS Platform for Research, Code Generation and Data Analysis for fulfillment of the requirement for the degree of B. Tech. in ARTIFICIAL INTELLIGENCE AND DATA SCIENCE and submitted to the Department of Artificial Intelligence and Data Science at Shah & Anchor Kutchhi Engineering College, Mumbai, is an authentic record of our own work carried out during the period from June 2025 to May 2026 under the supervision of Dr. Rahul Pachade

The matter presented in this dissertation/Project Report has not been submitted elsewhere in part or fully to any other University or Institute for the award of any other degree.

Signature of the students

Sr. No.	Name	Roll No.	Email ID	Signature
1	Shubh Shah	22UF17191AI053	shubh.17191@sakec.ac.in	
2	Anurag Kumar	22UF17647AI027	anurag.kumar17647@sakec.ac.in	
3	Shreyash Mokani	22UF17269AI032	shreyash.17269@sakec.ac.in	
4	Nigranth Shah	22UF17430AI051	nigranth.17430@sakec.ac.in	

Shah & Anchor Kutchhi Engineering College, Chembur

Date: 16/05/2026



Department of Artificial Intelligence and Data Science

(An Autonomous Institute Affiliated to University of Mumbai)

CERTIFICATE OF PLAGIARISM CHECK

CERTIFICATE OF PLAGIARISM CHECK	
Title of the Dissertation/Report	Engunity X : AI-Powered SaaS Platform for Research, Code Generation and Data Analysis
Name of the Supervisor/Guide	Dr. Rahul Pachade
Name of the Department	Department of Artificial Intelligence and Data Science
Similar content (%) identified	%
Acceptable Maximum Limit	10 %
Name of the Similarity tool used for Plagiarism Report	Turnitin
Date of Verification	07/05/2026
Name & Sign of the DAIP Member	
Name & Sign of Students	Shubh Shah, Anurag Kumar, Shreyash Mokani, Nigranth Shah
Name & Sign of the Guide	Dr. Rahul Pachade

Shah & Anchor Kutchhi Engineering College, Chembur

Date: 16/05/2026

ACKNOWLEDGEMENTS

We would like to express our heartfelt gratitude to all those who supported and guided us throughout this project. We sincerely thank our project guide, Dr.Rahul Pachade, for their valuable insights and constant encouragement. We are also grateful to the faculty of the Department of Artificial Intelligence and Data Science for providing us with the necessary resources and knowledge. Special thanks to our peers for their support and collaboration, and to our families for their unwavering support throughout this journey.

Sr. No.	Name	Roll No.	Email ID	Signature
1	Shubh Shah	22UF17191AI053	shubh.17191@sakec.ac.in	
2	Anurag Kumar	22UF17647AI027	anurag.kumar17647@sakec.ac.in	
3	Shreyash Mokani	22UF17269AI032	shreyash.17269@sakec.ac.in	
4	Nigranth Shah	22UF17430AI051	nigranth.17430@sakec.ac.in	

Shah & Anchor Kutchhi Engineering College, Chembur

Date: 16/05/2026

ABSTRACT

Engineering students and practitioners routinely operate across at least three disconnected environments: a retrieval pipeline for document queries, a code editor with a generative extension, and whatever external tool happens to be available for pressure-testing assumptions. We encountered this fragmentation first-hand during early development, and it drove the central question behind Engunity X—whether retrieval, execution, and adversarial reasoning could reinforce each other in a single coherent platform rather than exist as separate tools that the user stitches together manually.

The result is a five-service microarchitecture where all capabilities share infrastructure but are strictly process-isolated. *OmniRAG* first classifies each query's structural complexity using a fine-tuned DistilBERT router (trained on 18,500 labelled queries across four complexity tiers), then routes to the appropriate retrieval path: direct generation for simple lookups, hybrid FAISS + BM25 with Multi-Query expansion for single-hop factual queries, or GraphRAG community traversal with map-reduce synthesis for multi-hop and abstract questions. *Code Lab* validates AI-generated code through AST analysis and executes it inside a chroot-isolated process with 512 MB memory and 30-second wall-time caps, feeding errors back to the LLM for autonomous correction. *Decision Vault* audits argument quality against an Evidence Quality Score threshold and deliberately challenges reasoning that does not clear it, specifically countering the sycophancy that RLHF-trained models exhibit.

We evaluated OmniRAG on 500 benchmark queries drawn from a publicly available CS documentation corpus (10,000 labelled document chunks, fully separated from the classifier training set). On multi-hop and abstract queries, OmniRAG achieved $88.0\% \pm 2.4$ retrieval accuracy versus a standard single-stage FAISS baseline at $64.0\% \pm 3.1$, with MRR improving from 0.41 to 0.81 and F1 reaching 0.84. P95 end-to-end latency held below 500 ms under a sustained load of 500 concurrent users. Contextual compression reduced token volume by 68% while maintaining retrieval recall above 94%. The system addresses a gap in existing frameworks: no open platform currently combines query-adaptive retrieval with in-loop execution validation and adversarial decision feedback in a single deployable architecture.

Keywords: Retrieval-augmented generation, agentic AI, knowledge graph, secure code sandbox, adversarial decision reasoning, DistilBERT, engineering education, microservices

Contents

Certificate	ii
Candidate's Declaration	iii
Certificate of Plagiarism Check	iv
Dedication	v
Acknowledgements	v
Abstract	vi
1 Introduction	1
1.1 Background and Motivation	1
1.1.1 The Engineering Education Context	2
1.1.2 The Role of Large Language Models in Technical Assistance	3
1.2 Problem Statement and Objectives	4
1.2.1 Core Problem: Tool Fragmentation in Engineering Workflows	4
1.2.2 Five Specific Technical Challenges	4
1.2.2.1 Challenge 1 — Shallow AI Reasoning on Complex Queries	4
1.2.2.2 Challenge 2 — Unsecured and Non-Interactive Code Execution	5
1.2.2.3 Challenge 3 — Stateless Interaction Losing User Context	5
1.2.2.4 Challenge 4 — Multimodal Gaps Preventing Engineering Image Reasoning	5
1.2.2.5 Challenge 5 — Absence of Adversarial Decision Support	5
1.2.3 Project Objectives	5
1.3 Scope of the Project	6
1.4 Comparison with Existing Tools	6
1.5 Principal Contributions	7
1.6 Organisation of the Report	8
2 Literature Review	9
2.1 Retrieval-Augmented Generation: Foundations and Evolution	10

2.1.1	Naive RAG: The Foundational Paradigm	10
2.1.2	Advanced RAG Variants	10
2.1.2.1	HyDE: Hypothetical Document Embeddings	10
2.1.2.2	CRAG: Corrective Retrieval-Augmented Generation	11
2.1.2.3	GraphRAG: Graph-Based Community Summarization	11
2.1.3	Modular RAG and the Pipeline Paradigm	12
2.2	Autonomous Agents and Iterative Reasoning	12
2.2.1	The ReAct Framework	12
2.2.2	Multi-Agent Role Specialisation	13
2.2.3	Reflexion: Verbal Reinforcement Learning	14
2.2.4	Comparison of Agent Frameworks	14
2.3	Multimodal Perception Systems	14
2.3.1	CLIP: Cross-Modal Embedding	14
2.3.2	YOLOv8: Real-Time Object Detection	15
2.3.3	OCR for Engineering Handwriting	16
2.4	Secure Code Execution Infrastructure	16
2.4.1	Online Judge Infrastructure	16
2.4.2	Judge0 and Piston: Open Execution APIs	17
2.4.3	Monaco Editor and the LSP Ecosystem	17
2.5	Pedagogical and UX Considerations	18
2.5.1	Cognitive Load in Technology-Enhanced Learning	18
2.5.2	Intelligent Tutoring System Research	18
2.6	Research Gap Analysis	18
2.7	Summary	19

3 System Design and Architecture 20

3.1	Design Philosophy	20
3.2	Overall Architecture	20
3.2.1	Request Lifecycle	21
3.2.2	Database Architecture	22
3.3	The OmniRAG Pipeline	23
3.3.1	Design Rationale	23
3.3.2	Complexity Classification	23
3.3.3	Single-Hop Retrieval Flow	24
3.3.4	Multi-Hop (Graph RAG) Flow	25
3.3.5	Recursive Intensive Flow	25
3.4	The DeepResearchAgent	25
3.4.1	Architecture	25
3.4.2	Phase 1: Decompose	25

3.4.3	Phase 2: Search	26
3.4.4	Phase 3: Evaluate	26
3.4.5	Phase 4: Refine	26
3.4.6	Phase 5: Synthesize	27
3.5	Multimodal Visual Perception	27
3.5.1	Multimodal Processing Pipeline	27
3.6	Code Laboratory	28
3.6.1	Execution Security Model	28
3.6.2	Interactive Terminal via WebSocket	29
3.6.3	AI Inline Completion	29
3.6.4	Python Debugger via DAP	30
3.6.5	AI Error Correction Loop	30
3.7	Memory Architecture	30
3.7.1	Short-Term Session Memory	30
3.7.2	Long-Term User Profiles	31
3.8	Analytics Workbench	31
3.9	Decision Vault	31
3.10	Authentication and Security Architecture	32
3.11	Deployment Architecture	33
3.12	Frontend Architecture	33

4	Implementation Details	35
4.1	Backend Implementation	35
4.1.1	FastAPI Application Structure	35
4.1.2	OmniRAG Implementation Details	36
4.1.3	DistilBERT Complexity Classifier Training	36
4.1.4	FAISS Index Construction	37
4.1.5	Knowledge Graph Construction	38
4.2	Frontend Implementation	38
4.2.1	Next.js App Router Structure	38
4.2.2	SSE Stream Consumption	39
4.2.3	Monaco Editor Integration	39
4.2.4	XTerm.js Terminal Integration	40
4.3	Celery Background Workers	40
4.4	Testing Infrastructure	40
4.4.1	Backend Testing	40
4.4.2	Frontend Testing	41
4.4.3	E2E Test Scenarios	41
4.5	Deployment and DevOps	42

4.5.1	Docker Compose Configuration	42
4.5.2	CI/CD Pipeline	43
4.5.3	Monitoring and Observability	43
4.6	Security Implementation Details	44
4.6.1	JWT Authentication Flow	44
4.6.2	Rate Limiting Implementation	45
4.6.3	Input Sanitisation	45
4.7	Performance Optimisations	46
4.7.1	Async-First Architecture	46
4.7.2	Connection Pooling	46
4.7.3	Response Caching	46
4.7.4	Streaming Response Design	47
4.8	Known Issues and Workarounds	47
5	Results and Discussions	49
5.1	Evaluation Methodology	49
5.1.1	Evaluation Datasets	49
5.1.2	Metrics	49
5.2	Platform Screenshots: Visual Overview	50
5.3	OmniRAG Retrieval Evaluation	50
5.3.1	Overall Retrieval Performance	50
5.3.2	Per-Strategy Breakdown	52
5.3.3	Effect of CRAG Web Fallback	52
5.3.4	Effect of Contextual Compression	52
5.4	System Performance and Load Testing	53
5.4.1	Cold-Start Latency	54
5.5	Code Lab Evaluation	54
5.5.1	AI Error Correction Performance	54
5.6	Decision Vault Evaluation	54
5.6.1	EQS Calibration	54
5.7	Ablation Study	55
5.8	Qualitative Pilot Study	55
5.9	Discussion	56
5.9.1	Key Findings	56
5.9.2	Primary Limitations	57
6	Conclusions	58
6.1	Summary of Contributions	58
6.1.1	OmniRAG Pipeline	58
6.1.2	DeepResearchAgent	59

6.1.3	Multimodal Input Layer	59
6.1.4	Evidence Quality Score and Decision Vault	59
6.2	Objective Review	59
6.3	Limitations	60
6.3.1	Scalability Ceiling	60
6.3.2	Knowledge Graph Staleness	60
6.3.3	YOLOv8n Taxonomy Mismatch	61
6.3.4	Session Length Limit	61
6.4	Future Work	61
6.4.1	External Vector Store Migration	61
6.4.2	Engineering-Domain Fine-Tuning	62
6.4.3	Incremental Knowledge Graph Updates	62
6.4.4	Formal User Study	62
6.4.5	Test-Case-Augmented Error Correction	62
6.4.6	Mobile-Responsive Progressive Web App	63
6.5	Ethical Considerations	63
6.5.1	Data Privacy	63
6.5.2	AI Output Reliability	63
6.5.3	Academic Integrity	64
6.6	Final Remarks	64
Bibliography		66
A Appendices		67
A.1	Technology Stack Reference	67
A.2	Full API Contract Reference	67
A.2.1	OmniRAG Stream Endpoint	67
A.2.2	DeepResearch Stream Endpoint	68
A.2.3	Code Execution Endpoint	68
A.2.4	Analytics Endpoint	69
A.3	Environment Variable Reference	69
A.4	Database Schema Reference	70
A.4.1	Core PostgreSQL Tables	70
A.5	Glossary of Terms	71
A.6	List of Abbreviations	73

List of Figures

2.1	Literature Review	9
5.1	(a) Main Dashboard	51
5.2	(b) Data Analysis module	51
5.3	(c) Code Lab environment	51
5.4	(d) Document Q&A interface	51
5.5	EngUnity X frontend overview. (a) Unified dashboard; (b) AI-assisted CSV analysis; (c) Sandboxed Code Lab with Monaco editor; (d) OmniRAG-powered document retrieval.	51
5.6	(e) Research Agent interface	52
5.7	(f) Interactive Notebook	52
5.8	(g) Decision Vault interface	52
5.9	(h) Knowledge Hub	52
5.10	EngUnity X additional interfaces. (e) DeepResearchAgent with progress tracking; (f) AI-assisted interactive notebook; (g) Decision Vault with EQS scoring; (h) Document management and Knowledge Hub.	52

List of Tables

1.1	Project scope boundaries.	7
1.2	Feature comparison of Engunity X against existing AI-assisted engineering tools.	8
2.1	Evolution of RAG architectures and their corresponding Engunity implementation.	13
2.2	Comparison of autonomous agent frameworks and their influence on Engunity.	15
2.3	Research gap analysis: Engunity X vs. closely related prior work. . .	19
3.1	The four architectural planes, their technologies, and primary responsibilities.	21
3.2	Database allocation by data type.	22
3.3	OmniRAG complexity classes, strategies, and query characteristics. .	23
3.4	ResearchDepth parameter matrix controlling agent behaviour.	26
3.5	Security constraints applied to each Code Lab execution container. .	28
3.6	Evidence Quality Score (EQS) rubric.	32
3.7	Docker Compose services in the Engunity production stack.	33
4.1	OmniRAG pipeline modules and their implementations.	36
4.2	Celery background tasks and their trigger conditions.	41
4.3	Frontend test coverage by component category.	42
4.4	Rate limit configuration per endpoint group.	45
4.5	Known issues, root causes, and current workarounds.	48
5.1	Evaluation datasets used across all experiments.	50
5.2	Retrieval evaluation metrics: OmniRAG vs. standard RAG baseline. .	53
5.3	Retrieval accuracy broken down by OmniRAG routing strategy. . . .	53
5.4	Load test results: P95 latency, error rate, and throughput at four concurrency tiers.	54
5.5	Container cold-start latency per language in Code Lab.	54
5.6	AI error correction success rate by error category.	55
5.7	Decision Vault performance on 100 argument-evidence pairs.	55

5.8	Ablation study: impact of removing individual OmniRAG components.	56
5.9	Pilot study results (n=5 participants, 4 tasks each).	56
6.1	Review of project objectives against measured outcomes.	60
A.3	Backend environment variable reference.	69
A.4	Glossary of technical terms used in this report.	71
A.1	Complete backend technology stack with versions and purposes.	74
A.2	Complete frontend technology stack with versions and purposes.	75
A.5	List of abbreviations used in this report.	76





Chapter 1

Introduction

1.1 Background and Motivation

The last decade has witnessed an unprecedented convergence of artificial intelligence with nearly every domain of human endeavour. Engineering education—long characterised by its rigorous, problem-centric curriculum—has not been immune to this transformation. Generative language models, autonomous reasoning agents, and multimodal perception systems have all arrived at a pace that outstrips the ability of existing educational tooling to absorb them. The result is a situation that every engineering student and practitioner will recognise: a fragmented ecosystem of disconnected point solutions that each solve one narrow problem well but fail entirely to communicate with one another.

A typical session in an engineering project today might look like this. A student opens a browser to search for a relevant research paper, copy-pastes a code snippet from that paper into a local editor, runs the snippet against a dataset in a Jupyter notebook, then asks a general-purpose chatbot about a confusing output, and finally debates a design decision with a teammate whose counterargument they cannot independently verify. Five different tools, five different contexts, no persistent memory between any of them. Each context switch carries a cognitive penalty—a finding that is well-established in the educational psychology literature [1]—and the cumulative cost to learning efficiency is substantial.



The global EdTech market, valued at approximately USD 254 billion in 2021 and projected to exceed USD 605 billion by 2027, reflects the scale of this problem and the commercial appetite for solutions.

Yet the dominant products in this space remain either broad-purpose productivity tools (ChatGPT, Gemini, Copilot) that were not designed for engineering education specifically, or narrow vertical tools (Wolfram Alpha, MATLAB, Overleaf) that address only one aspect of the engineering workflow. No single platform combines document retrieval, code execution, autonomous research, and adversarial reasoning under a unified memory and authentication layer.

Engunity X was conceived to close this gap. It is a SaaS platform built around four interlocking capabilities—an adaptive retrieval-augmented generation (RAG) pipeline, an autonomous research agent, a secure sandboxed code laboratory, and an adversarial decision-auditing module—all sharing a common memory architecture, authentication layer, and streaming API. The platform was designed from first principles to be the single environment in which an engineering student can move from a vague research question to a verified, executed, and critically examined answer without ever leaving the interface.

1.1.1 The Engineering Education Context

Engineering undergraduate programmes in India typically span eight semesters, culminating in a major project in the seventh and eighth semesters. During this project, students are expected to survey relevant literature, propose and implement a system, evaluate it empirically, and document their findings. Each of these activities maps directly onto one of Engunity's four capabilities:

1. **Literature survey** → DeepResearchAgent with OmniRAG retrieval
2. **System implementation** → Code Lab with AI inline completion and debugging
3. **Empirical evaluation** → Analytics Workbench with AI-generated insights
4. **Critical analysis** → Decision Vault with Evidence Quality Score auditing

The alignment is intentional. Engunity was designed by engineering students, for engineering students, with direct knowledge of where the

friction points in a final-year project are. The Department of Artificial Intelligence and Data Science at Shah & Anchor Kutchhi Engineering College provided the real-world context and evaluation environment for this work.

1.1.2 The Role of Large Language Models in Technical Assistance

Large language models (LLMs) such as GPT-4 [2] have demonstrated remarkable zero-shot and few-shot performance on a wide range of technical tasks including code generation, mathematical reasoning, and scientific question answering. However, raw LLM capability does not translate directly into a reliable engineering assistant for several reasons:

- **Hallucination:** LLMs generate plausible-sounding but factually incorrect statements at a non-trivial rate [3]. In engineering contexts, a hallucinated equation or incorrect API signature can cause significant wasted effort.
- **Stale knowledge:** Models are trained on a snapshot of the web up to a cutoff date. Rapidly evolving fields such as deep learning see major framework changes every few months that a frozen model cannot reflect.
- **Context blindness:** A general-purpose chatbot has no knowledge of the user's specific project documents, codebase, or prior interactions unless explicitly provided in the prompt.
- **Sycophancy:** RLHF-trained models are prone to agreeing with user-presented arguments even when those arguments contain logical errors [4]. This is particularly dangerous in engineering design decisions.

Engunity addresses each of these failure modes directly. Retrieval-augmented generation grounds responses in retrieved documents. The CRAG web fallback mechanism overcomes knowledge staleness. The dual-layer memory system provides project-specific context across sessions. The Decision Vault's adversarial auditing module directly counters sycophancy by challenging arguments that fail an Evidence Quality Score threshold.

1.2 Problem Statement and Objectives

1.2.1 Core Problem: Tool Fragmentation in Engineering Workflows

The core problem Engunity addresses is **tool fragmentation**: the absence of a unified environment in which the cognitive activities of engineering work—retrieve, reason, code, analyse, decide—can be performed with shared context and persistent state. This fragmentation is not merely inconvenient; it has measurable learning consequences.

Sweller’s Cognitive Load Theory [1] distinguishes between *intrinsic load* (the inherent complexity of the material), *extraneous load* (effort caused by poor interface design), and *germane load* (effort that produces learning). Tool fragmentation drives up extraneous load without any corresponding benefit to germane load, directly reducing learning efficiency. Every manual copy-paste between a chatbot and an IDE, every re-explanation of context to a new tool, every search for a paper already reviewed—these are extraneous load events. Reducing them is the central engineering goal of Engunity X.

1.2.2 Five Specific Technical Challenges

The project addresses five concrete sub-problems, each well-defined enough to be evaluated independently:

1.2.2.1 Challenge 1 — Shallow AI Reasoning on Complex Queries

Standard RAG systems perform acceptably on factual single-hop questions but collapse on multi-document synthesis tasks. A student asking “Compare the memory management approaches of the three most common garbage-collected languages” needs multi-hop graph traversal and synthesis, not a simple nearest-neighbour document lookup. Existing platforms offer no mechanism to detect query complexity and adapt the retrieval strategy accordingly.

1.2.2.2 Challenge 2 — Unsecured and Non-Interactive Code Execution

General-purpose chatbots can generate code but cannot execute it. The few platforms that do execute code do not integrate the output back into a conversational AI pipeline for autonomous error correction. This leaves the student stranded between a code generator and a manual debugging session, with no intelligent bridge.

1.2.2.3 Challenge 3 — Stateless Interaction Losing User Context

Most AI assistants begin every conversation from scratch. There is no memory of the user's prior queries, the documents they have already reviewed, or the code they have already written. For a semester-long engineering project, this means the student must repeatedly re-establish context that should already be known to the system.

1.2.2.4 Challenge 4 — Multimodal Gaps Preventing Engineering Image Reasoning

Engineering workflows routinely involve circuit diagrams, flowcharts, handwritten derivations, and system architecture screenshots. General-purpose models that accept images (GPT-4V, Gemini) do not integrate visual understanding into a downstream RAG pipeline—images are processed in isolation and the extracted meaning is not retrievable in future queries.

1.2.2.5 Challenge 5 — Absence of Adversarial Decision Support

AI systems trained with reinforcement learning from human feedback (RLHF) are prone to validating whatever the user says rather than critically evaluating it. This sycophancy [4] is particularly harmful in engineering, where an over-confident but incorrect design can lead to significant project failures.

1.2.3 Project Objectives

In response to the five challenges above, this project established the following seven measurable objectives:

1. **OmniRAG Pipeline:** Design and implement a four-tier complexity-adaptive retrieval system trained on 18,500 labelled queries, achieving at least 85% retrieval accuracy on multi-hop queries.
2. **DeepResearchAgent:** Build an autonomous agent capable of iterative, multi-phase research with configurable depth tiers and a self-evaluated gap-filling loop.
3. **Multimodal Input Layer:** Develop a perception stack combining CLIP, YOLOv8n, and EasyOCR, integrated directly into the RAG prompt construction pipeline.
4. **Secure Code Laboratory:** Construct a containerized execution environment with Docker-enforced resource limits, a Monaco Editor IDE, WebSocket terminal, DAP debugger, and bidirectional AI-assisted error correction.
5. **Dual-Layer Memory:** Implement a persistence architecture combining short-term session summaries with long-term per-user profiles that persist across sessions.
6. **Decision Vault:** Deliver an adversarial reasoning module with a calibrated Evidence Quality Score (EQS) metric and structured counter-argument generation.
7. **Performance:** Achieve sub-500 ms P95 end-to-end latency under 500 concurrent users, validated by load testing.

1.3 Scope of the Project

Table 1.1 defines the boundary conditions of this project. Items outside scope represent genuine future work rather than implementation shortcuts.

1.4 Comparison with Existing Tools

Table 1.2 situates Engunity X relative to the most widely used AI-assisted tools in engineering education. No existing tool covers all six capability dimensions; Engunity X is the only platform that combines all of them in a single integrated environment with shared context and memory.

Table 1.1: Project scope boundaries.

In Scope	Out of Scope
Web-based SaaS platform (browser client)	Native mobile applications
Python, JavaScript, Go, Rust execution	Java, C#, R execution
Supabase + PostgreSQL authentication	LDAP / enterprise SSO
Single-instance Docker deployment	Kubernetes auto-scaling
English-language queries	Multi-lingual support
CS/Engineering domain documents	Medical, legal domain specialisation
FAISS in-process vector store	Distributed vector database (Qdrant)
Groq API cloud inference	Fine-tuned on-premise LLM hosting
PDF, DOCX, TXT, CSV ingestion	Audio and video ingestion

1.5 Principal Contributions

The principal novel contributions of this project are:

- **OmniRAG:** A four-tier complexity-adaptive retrieval pipeline with a DistilBERT classifier trained on 18,500 labelled queries, achieving 88% accuracy on multi-hop queries versus 64% for a standard RAG baseline—a 24-percentage-point improvement.
- **DeepResearchAgent:** A gap-analysis-driven iterative research loop with four configurable depth tiers (Quick, Standard, Deep, Exhaustive), integrating vector, graph, and live web retrieval in a single coherent agent.
- **Secure Execution with AI Feedback Loop:** A Docker-sandboxed code execution environment where execution errors and test failures are automatically fed back to the LLM for autonomous correction, reducing manual debugging time.
- **Evidence Quality Score (EQS):** A calibrated adversarial reasoning metric that quantifies argument quality on a four-point scale

Table 1.2: Feature comparison of Engunity X against existing AI-assisted engineering tools.

Tool	Doc RAG	Code Exec	Agent	Multimodal	Memory
ChatGPT 4o	✓	Limited	✓	✓	Session
GitHub Copilot	×	✓	×	×	File
Gemini 1.5	✓	Limited	✓	✓	Session
Perplexity AI	✓	×	Limited	×	×
Wolfram Alpha	×	✓	×	×	×
Replit AI	×	✓	×	×	Project
Engunity X	✓	✓	✓	✓	Long-term

and drives structured counter-argument generation, correctly flagging 78% of reasoning errors in evaluation.

- **Integrated Dual-Layer Memory:** A persistence system combining LLM-compressed session summaries with a long-term per-user profile that spans sessions, enabling genuinely context-aware assistance over the duration of a semester-long project.

1.6 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 reviews the technical lineages from which Engunity X draws, covering RAG architectures, autonomous agents, multimodal perception, secure sandboxing, and pedagogical positioning, closing with a research-gap analysis table. Chapter 3 provides a full technical specification of all five subsystems, including algorithmic pseudocode, data flow descriptions, database schema, API design, and security architecture. Chapter 5 presents the empirical evaluation including an ablation study, load-test results at four concurrency tiers, and a qualitative pilot study with five engineering students. Chapter 6 summarises the contributions, evaluates each objective against measured outcomes, discusses limitations in technical depth, and proposes six concrete directions for future work. The Appendices provide API contracts, database schema DDL, environment variable reference, a glossary of terms, and a list of abbreviations.

Chapter 2

Literature Review

AUTHOR	PUBLICATION DATE	RESEARCH ARTICLE	ARTICLE LINK
Patrick Lewis, Ethan Perez, Aleksandra Piktus et al.	2020	Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks	https://arxiv.org/abs/2005.11401
Ashish Vaswani, Noam Shazeer, Niki Parmar et al.	2017	Attention Is All You Need	https://arxiv.org/abs/1706.03762
Tom B. Brown, Benjamin Mann, Nick Ryder et al.	2020	Language Models Are Few-Shot Learners	Language Models are Few-Shot Learners
Michele Tufano, C. Watson, G. Bavota et al.	2019	An Empirical Study on Learning Bug-Fixing Patterns in the Wild via Neural Machine Translation	An Empirical Study on Learning Bug-Fixing Patches in the Wild via...

Figure 2.1: Literature Review

The design of Engunity X is grounded in four major technical lineages: retrieval-augmented generation and its advanced variants, autonomous reasoning agents, multimodal perception systems, and secure code execution infrastructure. This chapter surveys each lineage in depth, traces the key research contributions that directly influenced design decisions, and closes with a gap analysis table situating Engunity X relative to the most closely related prior work.

2.1 Retrieval-Augmented Generation: Foundations and Evolution

2.1.1 Naive RAG: The Foundational Paradigm

The retrieval-augmented generation paradigm was formalised by (author?) [5], who demonstrated that open-domain question answering could be substantially improved by combining a neural retriever (DPR [6]) with a sequence-to-sequence generator (BART). The retriever indexes a static Wikipedia dump and returns the top- k passages for a query; the generator conditions its output on both the query and the retrieved passages. On the Natural Questions and TriviaQA benchmarks, this approach outperformed GPT-3 (then the state of the art for closed-book QA) by substantial margins.

The core insight—that grounding generation in retrieved evidence reduces hallucination and enables the model to access knowledge beyond its training data—was immediately influential. Within two years of publication, RAG had become the dominant architectural pattern for knowledge-intensive NLP tasks, spawning a large ecosystem of variants addressing its limitations.

2.1.2 Advanced RAG Variants

The limitations of naive RAG are well-characterised in the literature: (1) retrieval quality is a hard ceiling on generation quality; (2) retrieved passages may be only loosely relevant to the actual information need; (3) the context window is finite and cannot accommodate all retrieved content. The research community has addressed each of these in turn.

2.1.2.1 HyDE: Hypothetical Document Embeddings

(author?) [7] observed that the embedding of a short factual query is often geometrically far from the embedding of a long, detailed document passage that answers it—the query and the answer live in different parts of the embedding space. Their solution, HyDE (Hypothetical Document Embeddings), uses the LLM to generate a hypothetical answer document from the query, then retrieves based on the embedding of that hypothetical rather than the original query. Since both the hypotheti-

cal and the real passage are long, expository texts, their embeddings are much closer. On several BEIR benchmarks, HyDE improved nDCG@10 by 5–10 percentage points over standard dense retrieval.

Engunity’s OmniRAG pipeline implements HyDE in the single-hop retrieval path. For each incoming query, the query transformation module generates a hypothetical answer and uses its embedding as an additional retrieval signal alongside the original query embedding. This is particularly effective for engineering documentation queries where the technical vocabulary of a user question (“how do I avoid race conditions in Python asyncio”) differs substantially from the vocabulary of the answer passages.

2.1.2.2 CRAG: Corrective Retrieval-Augmented Generation

(author?) [8] identified a critical failure mode in standard RAG: when the retrieved documents are poor—either off-topic or self-contradictory—the generator produces unreliable output without any signal that the retrieval failed. CRAG addresses this by inserting a lightweight evaluator between retrieval and generation. The evaluator assigns each retrieved document a relevance confidence score; if the maximum confidence falls below a threshold, the system triggers a web-search fallback rather than proceeding with the low-quality local retrieval.

In Engunity, CRAG’s confidence evaluation is implemented in the `crag_evaluator` module. The evaluator uses a cross-attention scorer on the query-document pair, with a threshold of 0.6 for local retrieval confidence. When a query’s best local document falls below this threshold, the pipeline automatically queries the Brave Search API, processes the results, and treats the top web pages as additional retrieved documents. This mechanism is transparent to the user; the source of the retrieved information (local or web) is indicated in the response metadata.

2.1.2.3 GraphRAG: Graph-Based Community Summarization

(author?) [9] proposed GraphRAG to address the fundamental limitation of vector RAG on queries that require synthesis across many documents—“what are the main themes in this corpus?” type questions. Standard RAG returns isolated passages; it cannot synthesise across them. GraphRAG builds a knowledge graph from the document

corpus, partitions the graph into communities using the Louvain algorithm, generates a community summary for each partition, and then performs query-focused retrieval over these community summaries using a map-reduce pattern.

Engunity adopts GraphRAG’s community-summary approach for the multi-hop retrieval path. The knowledge graph is built from the ingested document corpus using entity extraction via SpaCy’s named entity recogniser and relation extraction via a fine-tuned token classifier. Community detection uses the python-louvain library. Community summaries are generated offline by a background Celery worker and stored in MongoDB for fast retrieval at query time. This pre-computation means the map-reduce synthesis step at query time is fast, at the cost of graph staleness when documents are added—a limitation discussed in Chapter 5.

2.1.3 Modular RAG and the Pipeline Paradigm

The modular RAG framework [10] reconceptualised RAG as a configurable pipeline of interchangeable modules (query rewriter, retriever, re-ranker, compressor, generator, evaluator) rather than a fixed architecture. This framework is important for Engunity because it provides a principled basis for the OmniRAG routing logic: different query complexity tiers activate different subsets of the available modules.

Table 2.1 summarises the evolution of the RAG paradigm and identifies the Engunity component that implements each advance.

2.2 Autonomous Agents and Iterative Reasoning

2.2.1 The ReAct Framework

(author?) [11] introduced ReAct (Reason+Act), a prompting strategy that interleaves *reasoning traces* (chain-of-thought steps that make the model’s thinking explicit) with *actions* (API calls, search queries, code execution) and *observations* (the results of those actions). The interleaving is crucial: reasoning informs action selection, and observations update the reasoning. On HotpotQA, Fever, and AlfWorld

Table 2.1: Evolution of RAG architectures and their corresponding Engunity implementation.

Paradigm	Key Advance	Reference	Engunity Module
Naive RAG	Dense retrieval + generation	(author?) [5]	OmniRAG (baseline)
HyDE	Hypothetical doc embedding	(author?) [7]	Query Transformer
CRAG	Confidence-gated web fallback	(author?) [8]	CRAG Evaluator
GraphRAG	Community-summary map-reduce	(author?) [9]	Graph Retriever
Modular RAG	Pipeline composition	(author?) [10]	OmniRAG Router

benchmarks, ReAct substantially outperformed both chain-of-thought reasoning alone and task-specific supervised models.

Engunity’s DeepResearchAgent is built on the ReAct loop. Each research iteration generates a reasoning trace (which sub-questions are still unanswered?), selects an action (vector search / web search / graph traversal), executes it, and updates its evidence pool with the observation. The trace is streamed to the user as structured events, giving real-time visibility into the agent’s reasoning process.

2.2.2 Multi-Agent Role Specialisation

(author?) [12] introduced AutoGen, a framework in which multiple LLM-backed agents with distinct system prompts collaborate in a shared conversation to solve complex tasks. Role specialisation—assigning a planner agent, an executor agent, and a critic agent—was shown to substantially outperform single-agent approaches on software engineering and data analysis tasks.

Engunity adopts the multi-agent pattern in the DeepResearchAgent’s synthesis phase. A **PlannerAgent** decomposes the research query into typed sub-questions. A **SearchAgent** executes retrieval. A **CritiqueAgent**

(the Reflexion-inspired **SelfCritique** module) evaluates the draft report for logical gaps and redundancy before finalising.

2.2.3 Reflexion: Verbal Reinforcement Learning

(author?) [13] proposed Reflexion, in which an agent generates verbal feedback on its own failed actions and stores this feedback in a short-term memory buffer. On subsequent attempts at the same task, the agent is conditioned on its prior reflections, allowing it to avoid repeating mistakes without gradient-based fine-tuning. Reflexion **achieved state-of-the-art performance on HumanEval coding tasks** and WebArena web interaction tasks.

Engunity's **SelfCritique** module implements Reflexion's self-evaluation loop. After each OmniRAG response is generated, the module rates the response on three dimensions (relevance, completeness, factual groundness) on a 0–3 scale. **If the aggregate score falls below a threshold of 1.8, the response is passed back to the generation step with the critique attached as a system instruction.** This loop can run up to two iterations per query; empirically, one critique iteration was sufficient in over 90% of cases.

2.2.4 Comparison of Agent Frameworks

Table 2.2 compares the major open-source agent frameworks and their relevance to Engunity's design.

2.3 Multimodal Perception Systems

2.3.1 CLIP: Cross-Modal Embedding

(author?) [14] introduced **CLIP (Contrastive Language-Image Pre-training)**, which trains **a vision encoder (ViT) and a text encoder jointly using a contrastive objective** on 400 million **image-text pairs** scraped from the internet. **The result is a shared 512-dimensional embedding space** in which semantically related images and text descriptions are geometrically close. CLIP demonstrated remarkable zero-shot classification performance across 30+ computer vision benchmarks, often matching or exceeding task-specific supervised baselines.

Table 2.2: Comparison of autonomous agent frameworks and their influence on Engunity.

Framework	Key Feature	Limitation	Engunity Adoption
LangChain	Tool-calling, chain composition	Overhead, abstraction leakage	Partial (low-level primitives)
LlamaIndex	Structured data agents	Limited web search	RAG indexing patterns
AutoGen	Multi-agent conversation	Complex orchestration	Role specialisation pattern
Reflexion	Verbal self-correction	Requires task-specific evaluator	SelfCritique module
ReAct	Reason+Act interleaving	Single-agent only	DeepResearchAgent core loop

For Engunity, the critical property of CLIP is not classification performance but shared embedding space. When a user uploads an image of a circuit diagram, Engunity embeds it into the same 512-dimensional space used for text embeddings. This means the FAISS index—which already contains text chunk embeddings—can retrieve text passages that are semantically relevant to the image using a single unified query. This “one-index, multi-modal” architecture avoids the complexity of separate image and text retrieval pipelines.

2.3.2 YOLOv8: Real-Time Object Detection

(author?) [15] released YOLOv8, the eighth iteration of the YOLO (You Only Look Once) family of real-time object detection models. YOLOv8n (the “nano” variant used in Engunity) achieves 37.3 mAP at 80.4 FPS on the COCO dataset using only 3.2M parameters, making it suitable for deployment on CPU without a dedicated GPU.

Engunity uses YOLOv8n to detect and label objects within uploaded images. Detected object labels are incorporated into the visual context string passed to the RAG pipeline. This provides an additional signal beyond CLIP embeddings—while CLIP captures holistic semantic sim-

ilarity, YOLOv8 provides specific, symbolic object identifiers that can be matched against technical documentation.

2.3.3 OCR for Engineering Handwriting

EasyOCR [?] is an open-source multi-language OCR engine built on a CRNN (Convolutional Recurrent Neural Network) architecture. It supports 80+ languages and handles both printed and handwritten text with layout information. For engineering education contexts, handwritten derivations, annotated diagrams, and whiteboard photos are common inputs that require OCR to extract machine-readable content.

Engunity uses EasyOCR with layout-preserving output: detected text regions are returned with their bounding box coordinates, and the extracted text is concatenated with spatial markers (“top-left: [x1,y1], bottom-right: [x2,y2]”) before being included in the visual context string. This preserves spatial relationships between text elements, which is important for interpreting matrix equations and multi-line derivations where column alignment carries meaning.

2.4 Secure Code Execution Infrastructure

2.4.1 Online Judge Infrastructure

The secure sandboxing problem—allowing untrusted code to execute in a shared environment without compromising the host system—has been studied extensively in the context of online judge (OJ) systems such as Codeforces, LeetCode, and HackerRank. Common sandboxing approaches include:

- **chroot jails:** Restrict the process’s view of the filesystem to a subdirectory.
- **Linux namespaces:** Isolate process, network, and mount namespaces using Linux kernel primitives.
- **cgroups:** Enforce CPU, memory, and I/O resource limits at the kernel level.
- **seccomp:** Filter system calls allowed to the sandboxed process.

- **Docker:** A higher-level abstraction combining namespaces, cgroups, and overlay filesystems.

2.4.2 Judge0 and Piston: Open Execution APIs

Judge0 [?] is an open-source code execution API that supports 60+ programming languages and implements multi-layer sandboxing (Docker + isolate). Piston [?] is a similar open-source execution engine that uses Docker with fine-grained resource limits and supports arbitrary runtimes. Both systems are designed for competitive programming judge use cases and prioritise security and throughput over interactive debugging.

Engunity's Code Lab takes a different approach. Rather than using an existing execution API, it implements a custom `SecureSandbox` class in Python that manages Docker container lifecycles directly via the Docker SDK. This decision was driven by the need for *interactive* execution—the container must remain alive and responsive for WebSocket-based terminal interaction, which batch-oriented APIs like Judge0 and Piston are not designed for.

2.4.3 Monaco Editor and the LSP Ecosystem

Monaco Editor is the in-browser code editor powering Visual Studio Code. It supports syntax highlighting for 80+ languages, intelligent code completion via the Language Server Protocol (LSP), inline decorations for errors and warnings, and a rich API for programmatic control. Its WebAssembly-based implementation allows it to run entirely in the browser without a server-side LSP process.

Engunity uses Monaco v0.50 as the Code Lab IDE frontend. AI-assisted code completion is implemented as a custom Monaco completion provider that queries the OmniRAG pipeline rather than a traditional LSP server, allowing completions to incorporate knowledge from the user's uploaded documents alongside the current file content.

2.5 Pedagogical and UX Considerations

2.5.1 Cognitive Load in Technology-Enhanced Learning

Sweller's Cognitive Load Theory [1] has been widely applied in the design of intelligent tutoring systems and educational software. The key design principle it implies is: reduce extraneous load (caused by poor interface design and unnecessary complexity) to free cognitive capacity for germane load (the effort that actually produces learning). Engunity's design decisions consistently prioritise this principle.

The single unified chat interface, rather than separate query interfaces for each subsystem, reduces the extraneous load of deciding which tool to use. The automatic query complexity classification removes the need for the user to explicitly select a retrieval strategy. The persistent memory system eliminates the extraneous load of re-establishing context at the start of each session.

2.5.2 Intelligent Tutoring System Research

Intelligent Tutoring Systems (ITS) [16] personalise instruction based on a model of the learner's current knowledge state. Engunity's long-term user profile can be seen as a simplified learner model: it tracks which topics the user has queried most frequently, which documents they have revisited, and which complexity tiers their queries tend to fall into. This profile is used by the query rewriter to adapt explanations to the user's apparent expertise level.

2.6 Research Gap Analysis

Table 2.3 positions Engunity X relative to the eight most closely related prior works across six evaluation dimensions.

The gap analysis reveals that no prior work simultaneously addresses all six dimensions. The closest prior work is AutoGen [12], which covers agents and sandboxed execution, but lacks adaptive retrieval, knowledge graphs, multimodal input, and persistent memory. Engunity X is, to the best of our knowledge, the first open-source platform to integrate all six

Table 2.3: Research gap analysis: Engunity X vs. closely related prior work.

Work	Adapt.	Graph	Agents	Multimod.	Sandbox	Memory
Lewis et al. [5]	×	×	×	×	×	×
Yan et al. [8]	✓	×	×	×	×	×
Edge et al. [9]	×	✓	×	×	×	×
Yao et al. [11]	×	×	✓	×	×	×
Wu et al. [12]	×	×	✓	×	✓	×
Shinn et al. [13]	×	×	✓	×	×	✓
Radford et al. [14]	×	×	×	✓	×	×
Gao et al. [7]	×	×	×	×	×	×
Engunity X (ours)	✓	✓	✓	✓	✓	✓

Columns: *Adapt.* = adaptive routing; *Graph* = knowledge graph retrieval; *Agents* = autonomous agents; *Multimod.* = multimodal input; *Sandbox* = secure code execution; *Memory* = long-term user memory.

capabilities in a single deployable architecture targeted at engineering education.

2.7 Summary

This chapter has surveyed the four technical lineages that inform Engunity X's design. The RAG literature has evolved from naive dense retrieval to modular, adaptive pipelines; Engunity incorporates CRAG, HyDE, and GraphRAG as specific instances within its OmniRAG router. The autonomous agents literature has produced ReAct, AutoGen, and Reflexion, all of which influence the DeepResearchAgent's architecture. CLIP, YOLOv8n, and EasyOCR form the multimodal perception stack. Docker-based sandboxing with WebSocket interactivity is the Code Lab's execution model, advancing beyond static batch-execution APIs. The research gap analysis confirms that Engunity X's integrated six-capability architecture is novel in the literature. Chapter 3 now describes how each of these components is designed and implemented in detail.



Chapter 3

System Design and Architecture

3.1 Design Philosophy

Before detailing each subsystem, it is worth articulating the guiding principles behind Engunity’s architecture. The most common failure mode in student AI projects is what might be called “glue-code” architecture: take an LLM API, wrap a thin web layer around it, and call it a platform. That approach produces something that technically works but breaks down under real-world load, is impossible to diagnose when it produces bad outputs, and offers no foundation for improvement.

Engunity was designed around three principles: **traceability** (every module has defined inputs, outputs, and failure modes), **replaceability** (components communicate through typed interfaces so that any one can be swapped), and **observability** (every pipeline stage emits structured streaming events that the frontend can display in real time). These principles translate directly into architectural decisions: strict plane separation, Server-Sent Events (SSE) for streaming, Pydantic schemas for all API contracts, and Celery for background job isolation.

3.2 Overall Architecture

Engunity is organised into four planes that communicate only through typed API or message-queue contracts.

The client (Next.js PWA) communicates exclusively with the Orchestration Plane over HTTPS. The Inference and Persistence planes are never directly accessible from the browser. Background jobs—file indexing, bulk analysis, knowledge-graph community building—go through

Table 3.1: The four architectural planes, their technologies, and primary responsibilities.

Plane	Technology	Responsibilities
Presentation	Next.js 16, React 18, Tailwind CSS 3	Rendering, routing, real-time SSE consumption, Monaco IDE
Orchestration	FastAPI 0.115, Uvicorn, Celery 5.4, Redis	Auth, rate-limiting, job dispatch, API gateway
Inference	OmniRAG, AI agents, Groq API, Ollama	Retrieval, reasoning, generation, quality evaluation
Persistence	PostgreSQL, MongoDB, Redis, FAISS	Relational data, document store, cache, vector index

Celery, backed by Redis, so that long-running operations do not block the FastAPI event loop.

3.2.1 Request Lifecycle

A typical OmniRAG query follows this lifecycle:

1. The user sends a query via the Next.js frontend over HTTPS POST to `/api/v1/omni-rag/stream`.
2. FastAPI validates the JWT token against Supabase and checks the rate-limit counter in Redis.
3. The request is dispatched to the `OmniRAGPipeline.process_query()` coroutine.
4. The pipeline emits SSE events for each stage (metadata, content chunks, done/error).
5. The frontend consumes the event stream and updates the UI incrementally.
6. After completion, a background Celery task calls `memory_system.remember()` to persist the interaction.

This design ensures that the user sees the first token of the response within milliseconds of retrieval completing, rather than waiting for the full response to be generated.

3.2.2 Database Architecture

Engunity employs a polyglot persistence strategy, using each database engine for the data type it is best suited to.

Table 3.2: Database allocation by data type.

Engine	Version	Data Stored	Access Pattern
PostgreSQL	15	Users, sessions, projects, analytics	OLTP, relational queries
MongoDB	7.0	Document chunks, research reports, chat history	Document lookup by ID
Redis	7.2	Rate-limit counters, session cache, Celery broker	High-throughput KV
FAISS	1.8.0	Dense vector index (1536-dim)	ANN k-NN search

The key schema tables in PostgreSQL are:

- **users:** id, email, hashed_password, provider, created_at, profile_data (JSONB)
- **sessions:** id, user_id (FK), created_at, last_active, memory_summary (TEXT)
- **documents:** id, user_id (FK), filename, file_type, chunk_count, indexed_at
- **analytics_runs:** id, user_id (FK), dataset_name, row_count, result_json (JSONB)
- **decision_records:** id, user_id (FK), query, eqs_score, verdict, created_at
- **code_executions:** id, user_id (FK), language, exit_code, duration_ms, created_at

3.3 The OmniRAG Pipeline

3.3.1 Design Rationale

Early in development, Engunity had separate endpoints for simple questions, document search, and deep research. This worked but pushed a significant cognitive burden onto the student: they had to decide what *kind* of question they were asking before getting a useful answer. Empirical observation showed that students routinely underestimated query complexity and ended up on the wrong endpoint. The fix was to classify automatically and route internally. The pipeline exposes a single `process_query` method; the strategy is an implementation detail hidden from the user.

3.3.2 Complexity Classification

The `QueryComplexityClassifier` is a fine-tuned DistilBERT model (66M parameters) trained on 18,500 manually labelled queries across four complexity tiers. The training data was collected from three sources: (1) the Natural Questions dataset (simple and single-hop queries), (2) HotpotQA (multi-hop queries), and (3) custom engineering-domain queries annotated by the development team (recursive-intensive queries).

Table 3.3: OmniRAG complexity classes, strategies, and query characteristics.

Class	Strategy	Characteristics
SIMPLE	Direct generation	Conversational, definitional, no document dependency
SINGLE_HOP	Vector RAG + HyDE	Factual with clear document anchor, single retrieval topic
MULTI_HOP	Graph RAG map-reduce	Comparative, requires synthesis across multiple sources
RECURSIVE_INTENSIVE	Recursive chain-of-thought	Open-ended analytical, benefits from multi-step reasoning chains

The classifier achieves 91.3% macro-averaged F1 on a held-out test set of 2,000 queries. Misclassification is most common at the `SINGLE_HOP`

/ MULTI_HOP boundary, which is architecturally benign: routing a SINGLE_HOP query to the MULTI_HOP path costs slightly more compute but does not degrade answer quality.

3.3.3 Single-Hop Retrieval Flow

For standard single-hop queries, the pipeline executes the following multi-stage sequence:

1. **Memory & Multimodal Context:** Fetches user session summary and long-term profile from PostgreSQL. Processes attached images via CLIP (embedding) and YOLOv8n (detection).
2. **Query Transformation:** Rewrites the raw query for retrieval effectiveness; generates 3 query variants via multi-query expansion; generates a HyDE hypothetical answer document.
3. **Hybrid Retrieval:** Executes concurrent FAISS semantic search (top-20 chunks, cosine similarity) and BM25 keyword search (rank-bm25). Results are merged using a weighted combination score $s = \alpha \cdot s_{dense} + (1 - \alpha) \cdot s_{sparse}$ with $\alpha = 0.6$.
4. **Reranking & Filtering:** Applies FlashRank neural cross-attention reranking to deduplicate and select the top-10 most relevant chunks.
5. **CRAG Confidence Evaluation:** Scores each top chunk against the query using a cross-encoder. If maximum score < 0.6 , triggers Brave Search API web fallback, fetching and parsing up to 5 web pages.
6. **Contextual Compression:** Applies extractive token compression to reduce total context by 60–70% while preserving the most information-dense sentences.
7. **Response Generation & Self-Critique:** Generates a streaming response; runs SelfCritique evaluation; if score $< 1.8/3.0$, regenerates with critique attached.

3.3.4 Multi-Hop (Graph RAG) Flow

The multi-hop path retrieves knowledge-graph community summaries alongside standard vector chunks and synthesises them using a map-reduce pattern.

Map phase: The pipeline generates partial answers independently for each community summary and each top-5 document chunk, running all LLM calls concurrently via `asyncio.gather`. Each partial answer is a structured object with fields: `content`, `source_ids`, `confidence`.

Reduce phase: A synthesis prompt receives all partial answers and produces a single coherent final response. The synthesis prompt instructs the model to identify agreements, resolve contradictions, and cite specific sources for each claim.

3.3.5 Recursive Intensive Flow

Recursive queries expand the retrieval budget ($k = 40$ versus $k = 20$ for single-hop) and combine the larger document pool with knowledge-graph context. The assembled context is handed to `RecursiveReasoningAgent`, which implements a chain-of-thought loop where each step can spawn subsidiary inferences before committing to a final answer. Each intermediate step is emitted as a structured streaming event, giving the user partial visibility into the reasoning process.

3.4 The DeepResearchAgent

3.4.1 Architecture

The `DeepResearchAgent` is an autonomous agent that conducts iterative, multi-phase research over vector, web, and graph sources. It is implemented as an async state machine with the following states: `DECOMPOSE` → `SEARCH` → `EVALUATE` → `REFINE` → `SYNTHESIZE`.

3.4.2 Phase 1: Decompose

The `PlannerAgent` receives the user's research query and generates a set of typed sub-questions. Each sub-question is annotated with its

Table 3.4: ResearchDepth parameter matrix controlling agent behaviour.

Depth	Max Sub-Q	Max Iter	Min Src	Typical Use
QUICK	2	1	2	Quick factual look-up
STANDARD	5	3	4	Coursework research
DEEP	8	5	7	Thesis-level questions
EXHAUSTIVE	12	8	10	Full domain survey

type (factual, comparative, causal, or definitional) and its priority rank. The number of sub-questions is bounded by the `max_sub_questions` parameter of the chosen depth tier.

3.4.3 Phase 2: Search

For each sub-question, the **SearchAgent** executes three concurrent retrieval operations: (1) vector search via FAISS over the user’s uploaded documents; (2) web search via Brave Search API; (3) graph traversal over the knowledge graph community summaries. Results are deduplicated by URL/chunk ID and assembled into an evidence pool.

3.4.4 Phase 3: Evaluate

The **EvaluatorAgent** scores each evidence item on relevance to its sub-question using an LLM-based evaluator with a 0.4 threshold. Evidence items below threshold are discarded. The agent then identifies “gap” sub-questions: those for which the evidence pool contains fewer than `min_sources` items above threshold.

3.4.5 Phase 4: Refine

For each gap sub-question, the agent executes an additional search iteration, potentially broadening the query terms or switching sources (e.g., falling back to web search if local vector search was insufficient). This gap-filling loop runs until either all sub-questions are adequately evidenced or `max_iterations` is reached.

3.4.6 Phase 5: Synthesize

The `SynthesizerAgent` receives the complete evidence pool and generates a structured research report. The report format depends on the `output_format` parameter: `detailed` (full sections with citations), `summary` (executive summary), or `bullet_points` (key findings list). Each claim in the report is linked to its source evidence items via citation markers.

3.5 Multimodal Visual Perception

The `MultiModalVisionProcessor` wraps three perception components that together handle the visual inputs common in engineering education:

- **CLIP (ViT-B/32)**: Embeds each image into the same 512-dimensional space used for text embeddings, enabling cross-modal retrieval within the same FAISS index. Processing time per image: approximately 45 ms on CPU.
- **EasyOCR**: Extracts printed and handwritten text with layout information preserved. Detected text regions are concatenated with positional markers (bounding box coordinates) so that spatial queries remain meaningful.
- **YOLOv8n**: Detects objects and classifies them using the 80-class COCO taxonomy. Detected labels are included in the visual context string. Processing time: approximately 80 ms per image on CPU.

All three components produce text output that is concatenated into a visual context prefix, which is injected into the RAG prompt before retrieval. This architecture means the same pipeline handles text-only and multimodal queries without branching at the orchestration level.

3.5.1 Multimodal Processing Pipeline

When a user attaches an image, the following sequence executes:

1. Image is received as a base64-encoded string in the API request body.

2. The image is decoded and resized to 224×224 for CLIP and 640×640 for YOLOv8.
3. CLIP generates a 512-dimensional embedding vector.
4. YOLOv8n generates a list of detected objects with confidence scores > 0.45 .
5. EasyOCR extracts text regions with bounding boxes.
6. The three outputs are combined into a `VisualContext` object.
7. The visual context is prepended to the RAG query string.
8. The CLIP embedding is used as an additional query vector for FAISS retrieval.

3.6 Code Laboratory

3.6.1 Execution Security Model

The Code Lab is built around a `SecureSandbox` class that manages Docker containers for code execution. Each execution request spins up a fresh container instance with the following security constraints:

Table 3.5: Security constraints applied to each Code Lab execution container.

Constraint	Value
CPU quota	50% of one core (cgroup limit)
Memory limit	512MB (cgroup limit)
Wall-time limit	30 seconds (SIGKILL after timeout)
Network access	Disabled (<code>-network=none</code>)
Filesystem write	Ephemeral tmpfs only
Privileged mode	Disabled
Root execution	Disabled (runs as uid 1000)
Syscall filter	Docker default seccomp profile

The container image selection is language-dependent: `python:3.11-slim` for Python, `node:20-alpine` for JavaScript, `golang:1.22-alpine` for Go, and `rust:1.78-slim` for Rust. Each image is pre-pulled and cached on the host at startup to minimise container cold-start latency.

3.6.2 Interactive Terminal via WebSocket

The WebSocket endpoint `/ws/terminal/{project_id}` provides an interactive terminal through XTerm.js on the frontend. When a user opens a terminal session:

1. The backend starts a Docker container in interactive mode (`docker run -it`).
2. `stdin/stdout/stderr` are multiplexed over the WebSocket connection.
3. XTerm.js renders the terminal output with full ANSI escape code support.
4. The container remains alive until the user explicitly closes the terminal or a 10-minute inactivity timeout expires.

The same container instance used for code execution is reused for the terminal session, preserving the filesystem state (installed packages, created files) between execution and interactive exploration.

3.6.3 AI Inline Completion

Monaco Editor's completion API is extended with a custom `AICompletionProvider` that:

1. Captures the current file content, cursor position, and a 200-token sliding window around the cursor.
2. Includes a summary of the user's recently active files (from the project store).
3. Sends an asynchronous completion request to the OmniRAG pipeline.
4. Displays the response as Monaco's "ghost text" inline suggestion.

Completions are debounced with a 400 ms delay to avoid overwhelming the backend during rapid typing.

3.6.4 Python Debugger via DAP

A Debug Adapter Protocol (DAP) implementation using Python’s Bdb module supports breakpoints, step-over, step-into, step-out, and variable inspection for Python programs. DAP messages are serialised as JSON and transported over the existing WebSocket connection using a dedicated message type prefix (`dap://`).

3.6.5 AI Error Correction Loop

When a code execution returns a non-zero exit code, the error output is automatically fed back to the OmniRAG pipeline with the prompt: “The following Python code produced this error. Identify the root cause and provide a corrected version.” The corrected code is displayed as a diff in the Monaco editor, which the user can accept with a single click. In evaluation (see Chapter 5), this loop autonomously corrected 73% of syntax errors and 61% of common runtime errors within one correction iteration.

3.7 Memory Architecture

Two memory layers operate in parallel to provide continuity across queries and sessions.

3.7.1 Short-Term Session Memory

At natural pause points within a session (defined as 90 seconds of inactivity or a session reaching 20 turns), the LLM generates a hierarchical summary of the conversation history. This summary—rather than the raw message log—is passed as `memory_summary` on subsequent requests, keeping the context window lean while preserving the substance of prior exchanges.

The compression algorithm follows a two-level hierarchy: individual turn summaries (one sentence per turn) are first generated, then compressed into a session-level paragraph summary. Empirically, this reduces the token volume of the context window by 78% relative to passing the raw message history.

3.7.2 Long-Term User Profiles

After each OmniRAG interaction, `memory_system.remember()` stores a structured record in PostgreSQL containing: the original query, the generated response, the sources cited, the complexity class and strategy used, and the self-critique confidence score.

A background Celery worker runs every 6 hours, aggregating each user's interaction records into a profile that includes: most-queried topics, most-accessed documents, preferred response length, and average query complexity tier. This profile is consulted by the query rewriter to personalise the retrieval and generation process for each user.

3.8 Analytics Workbench

The Analytics Workbench handles structured data uploaded as CSV or XLSX files. The processing pipeline:

1. Pandas loads the file and performs automatic type inference (numeric, categorical, datetime).
2. Descriptive statistics are computed: mean, median, std, quartiles, missing-value count, correlation matrix.
3. Scikit-learn k-means clustering (k=3 default, elbow method for automatic k selection) identifies natural groupings.
4. Recharts visualisations are generated on the frontend: scatter plots, histograms, correlation heatmaps.
5. The OmniRAG pipeline generates a natural-language insight summary conditioned on the statistics.

The workbench supports datasets up to 100,000 rows and 500 columns without batching. Larger datasets trigger a Celery background job and notify the user on completion.

3.9 Decision Vault

The Decision Vault is the adversarial reasoning module. It implements the Evidence Quality Score (EQS) on a four-point scale:

Table 3.6: Evidence Quality Score (EQS) rubric.

Score	Label	Criteria
0	Unsupported	Claim made with no cited evidence
1	Weak	Evidence cited but tangentially related
2	Adequate	Evidence clearly relevant but incomplete
3	Strong	Multiple converging independent sources

When a user submits a decision rationale, the vault: (1) extracts individual claims using an NLP claim detector; (2) scores each claim against the EQS rubric; (3) computes an aggregate score as the mean of individual claim scores; (4) if aggregate < 1.8 , generates a structured counter-argument targeting the weakest-scored claim. This adversarial response is framed as a devil’s advocate challenge, not a refutation, to encourage reflection rather than defensiveness.

3.10 Authentication and Security Architecture

Engunity implements a dual-layer authentication system. Supabase handles OAuth2 flows (Google, GitHub) and issues JWT tokens with a 1-hour expiry. A local PostgreSQL user table maintains relational integrity (session records, document ownership, analytics history). FastAPI middleware validates every request against the JWT signature using the Supabase public key.

Security controls include:

- **Rate limiting:** Redis counters enforce per-user limits of 60 OmniRAG requests/minute and 20 Code Lab executions/minute.
- **SQL injection prevention:** All database queries use SQLAlchemy parameterised statements. No raw SQL string concatenation is used anywhere in the codebase.
- **Command injection prevention:** Code Lab execution uses Docker SDK API calls rather than shell subprocesses, eliminating shell injection vectors.
- **File upload validation:** Uploaded files are validated for MIME type and scanned for archive bombs (zip bombs) before processing.



- **Supabase RLS:** Row-Level Security policies ensure that each user can only access their own documents, sessions, and analytics records.

3.11 Deployment Architecture

The production deployment is specified as a Docker Compose stack with five services:

Table 3.7: Docker Compose services in the Engunity production stack.

Service	Image	Role
backend	Custom FastAPI image	API server, OmniRAG, agents
frontend	Custom Next.js image	SSR web application
postgres	postgres:15-alpine	Relational database
redis	redis:7.2-alpine	Cache and Celery broker
celery	Custom FastAPI image	Background worker processes

MongoDB and FAISS are run as external services (MongoDB Atlas and a host-mounted FAISS index directory respectively) to keep the Compose stack self-contained. Supabase is used as an external managed service for authentication.

3.12 Frontend Architecture

The Next.js frontend follows the App Router pattern introduced in Next.js 13. The route structure is:

- `/` — Landing page with feature overview and authentication
- `/chat` — Main OmniRAG conversational interface
- `/research` — DeepResearchAgent interface with depth selector
- `/code` — Code Lab with Monaco editor and terminal
- `/analytics` — Analytics Workbench with file upload and charts
- `/vault` — Decision Vault interface
- `/settings` — User profile, memory management, API key configuration

Global state is managed by Zustand stores (one per feature area). HTTP calls use Axios with a request interceptor that attaches the JWT token and a response interceptor that handles 401 token-refresh flows. Framer Motion provides page transition animations and micro-interactions. All API streaming responses (SSE) are consumed using the native EventSource API with a custom React hook that manages the stream lifecycle.

Chapter 4

Implementation Details

4.1 Backend Implementation

4.1.1 FastAPI Application Structure

The backend is organised as a Python package rooted at `backend/app/`. The package hierarchy is:

- `app/api/v1/` — Router modules for each feature area
- `app/agents/` — `DeepResearchAgent` and sub-agent classes
- `app/core/` — Configuration, logging, middleware, security
- `app/models/` — SQLAlchemy ORM model definitions
- `app/schemas/` — Pydantic request/response schemas
- `app/services/` — Business logic and external service clients
- `app/workers/` — Celery task definitions
- `app/utils/` — Shared utility functions

The `app/main.py` entry point creates the FastAPI application, registers all routers with their prefixes, configures CORS middleware using the `CORS_ORIGINS` environment variable, attaches the JWT authentication middleware, and starts the Celery worker process.

4.1.2 OmniRAG Implementation Details

The OmniRAG pipeline is implemented in `app/services/omni_rag/pipeline.py`. The core class is `OmniRAGPipeline`, instantiated as a singleton at application startup to avoid repeated model loading. The pipeline loads the DistilBERT complexity classifier, FAISS index, BM25 index, sentence-transformer model, CLIP model, and YOLOv8n model at init time. Each component is loaded lazily on first use if not present at startup.

The `process_query` method is the main entry point. It is implemented as an `async` generator that yields `StreamEvent` Pydantic objects. The FastAPI route handler wraps this generator in an `EventSourceResponse` (from `sse-starlette`), which serialises each event as a Server-Sent Event chunk.

Table 4.1: OmniRAG pipeline modules and their implementations.

Module	File	Key Classes
Complexity Classifier	<code>classifier.py</code>	<code>QueryComplexityClassifier</code>
Query Transformer	<code>transformer.py</code>	<code>QueryTransformer</code> , <code>HyDEGenerator</code>
Vector Retriever	<code>vector_retriever.py</code>	<code>FAISSRetriever</code>
BM25 Retriever	<code>bm25_retriever.py</code>	<code>BM25Retriever</code>
Graph Retriever	<code>graph_retriever.py</code>	<code>GraphRAGRetriever</code>
CRAG Evaluator	<code>crag_evaluator.py</code>	<code>CRAGEvaluator</code>
Context Compressor	<code>compressor.py</code>	<code>ContextualCompressor</code>
Response Generator	<code>generator.py</code>	<code>StreamingGenerator</code>
Self-Critique	<code>critique.py</code>	<code>SelfCritique</code>
Memory System	<code>memory.py</code>	<code>DualLayerMemory</code>

4.1.3 DistilBERT Complexity Classifier Training

The complexity classifier was trained using the HuggingFace `transformers` library. The training procedure:

1. **Dataset Construction:** 18,500 queries were collected from Natural Questions (simple/single-hop), HotpotQA (multi-hop), and a custom engineering annotation set (recursive-intensive). Labels were assigned: `SIMPLE=0`, `SINGLE_HOP=1`, `MULTI_HOP=2`, `RECURSIVE=3`.

2. **Tokenisation:** Queries were tokenised using `DistilBertTokenizerFast` with max length 128, padding to maximum sequence length in each batch.
3. **Model:** `DistilBertForSequenceClassification` with 4 output labels, pretrained weights from `distilbert-base-uncased`.
4. **Training:** AdamW optimiser, learning rate 2×10^{-5} , batch size 32, 5 epochs, linear warmup schedule (10% of steps), weight decay 0.01.
5. **Evaluation:** Macro-averaged $F1 = 0.913$ on the held-out 2,000-query test set.

The trained model weights are saved to `backend/app/models/complexity_class` and loaded at startup via `pipeline('text-classification', model=model_path)`

4.1.4 FAISS Index Construction

Documents are indexed when users upload them via `POST /api/v1/documents/upload`. The indexing pipeline:

1. The uploaded file (PDF/DOCX/TXT) is parsed using `pdfplumber` (PDF) or `python-docx` (DOCX).
2. The extracted text is split into chunks of 512 tokens with 50-token overlap using a sliding window chunker.
3. Each chunk is embedded using `sentence-transformers/all-mpnet-base-v2` (768-dimensional embeddings).
4. Embeddings are added to an `IndexFlatIP` (inner product / cosine similarity after L2 normalisation) FAISS index.
5. Chunk text and metadata (source document ID, chunk index, page number) are stored in MongoDB.
6. A BM25 index is updated with the new chunk texts.

The FAISS index is persisted to disk after each upload using `faiss.write_index()` and reloaded on server restart. The index file path is configurable via the `FAISS_INDEX_PATH` environment variable.

4.1.5 Knowledge Graph Construction

The knowledge graph is built by a Celery background task `build_knowledge_graph` that runs when triggered after document upload or on the 6-hour schedule. The construction pipeline:

1. **Entity Extraction:** SpaCy `en_core_web_sm` pipeline extracts named entities (PERSON, ORG, GPE, PRODUCT, WORK_OF_ART) from each document chunk.
2. **Co-occurrence Graph:** Entities that appear within the same chunk are connected with an edge weighted by co-occurrence frequency.
3. **Relation Extraction:** For each entity pair, a simple dependency-parse-based relation extractor identifies the predicate verb connecting them.
4. **Community Detection:** The Louvain algorithm (`python-louvain`) partitions the entity graph into communities that maximise modularity.
5. **Community Summarisation:** For each community, the LLM generates a natural-language summary of the entities and relations. These summaries are stored in MongoDB.

4.2 Frontend Implementation

4.2.1 Next.js App Router Structure

The frontend uses Next.js 16's `App Router`, where each route is a directory containing a `page.tsx` file. The global layout (`app/layout.tsx`) provides the authentication wrapper, theme provider, and Zustand store initialiser. Route-specific layouts (`app/(dashboard)/layout.tsx`) provide the sidebar navigation and header.

State management uses Zustand stores scoped by feature:

- `useAuthStore` — Authentication state (user, token, login/logout actions)
- `useChatStore` — OmniRAG session, messages, streaming state

- `useResearchStore` — DeepResearchAgent query, depth, progress, report
- `useCodeStore` — Active file, language, execution output, terminal state
- `useAnalyticsStore` — Uploaded dataset, statistics, cluster results

4.2.2 SSE Stream Consumption

Server-Sent Events are consumed using a custom React hook `useSSEStream`. The hook:

1. Creates a `fetch` request with `method: 'POST'` and the query payload in the body (the native `EventSource` API does not support POST requests).
2. Reads the response body as a `ReadableStream` using the Streams API.
3. Decodes each chunk as UTF-8 and splits on the SSE line protocol (`data: <json>`).
4. Dispatches each parsed event to the appropriate Zustand store action.
5. Handles reconnection on network errors with exponential backoff (max 3 retries).

4.2.3 Monaco Editor Integration

The Code Lab uses Monaco Editor v0.50, integrated via the `@monaco-editor/react` package. Key integration points:

- **Language detection:** File extension is mapped to Monaco language ID (e.g., `.py` → `python`).
- **Theme:** A custom dark theme `engunity-dark` is registered at startup with Engunity's colour palette.
- **AI completion provider:** A `CompletionItemProvider` is registered that calls the backend `/api/v1/code/complete` endpoint on each trigger character or `Ctrl+Space`.

- **Inline decorations:** Execution errors are decorated as red underlines using Monaco's `setModelMarkers` API.
- **Diff view:** AI-corrected code is shown as a diff using Monaco's `DiffEditor` component.

4.2.4 XTerm.js Terminal Integration

The interactive terminal uses XTerm.js v5.3. The terminal component:

1. Creates an XTerm `Terminal` instance with UTF-8 encoding and 1000-line scroll buffer.
2. Connects to the backend WebSocket endpoint `/ws/terminal/{project_id}`.
3. Forwards XTerm's `onData` events (keyboard input) to the WebSocket as raw bytes.
4. Writes WebSocket messages (terminal output) to XTerm using `terminal.write`.
5. Attaches the FitAddon to resize the terminal dimensions on window resize.

4.3 Celery Background Workers

Background tasks are defined in `app/workers/tasks.py` and registered with the Celery application configured to use Redis as both broker and result backend.

4.4 Testing Infrastructure

4.4.1 Backend Testing

The backend test suite uses `pytest` with the following structure:

- `tests/unit/` — Isolated tests for individual modules (classifier, compressor, memory)
- `tests/integration/` — Tests for API endpoints with a test PostgreSQL database and mocked external services

Table 4.2: Celery background tasks and their trigger conditions.

Task	Trigger	Description
<code>index_document</code>	Document upload	Chunk, embed, index to FAISS and BM25
<code>build_knowledge_graph</code>	Post-index or schedule	Entity extraction, graph construction, community summaries
<code>update_user_profile</code>	Every 6 hours	Aggregate interaction logs into long-term profile
<code>compress_session</code>	90s inactivity or 20 turns	Hierarchical session summary compression
<code>cleanup_containers</code>	Every 10 minutes	Remove expired Docker containers
<code>export_analytics_pdf</code>	Analytics export request	Generate PDF report from analytics results

- `tests/e2e/` — End-to-end tests using Playwright that drive the full stack

Key test fixtures include: a SQLite in-memory database (replacing PostgreSQL for unit/integration tests), a mock Groq client that returns deterministic responses, and a mock FAISS index pre-loaded with 100 test document chunks.

The test suite achieves 78% line coverage across the backend codebase, with the lowest coverage in the multimodal processing modules (58%) due to the difficulty of mocking CLIP and YOLOv8 inference in CI environments.

4.4.2 Frontend Testing

Frontend testing uses Vitest for unit/component tests and Playwright for E2E tests.

4.4.3 E2E Test Scenarios

The Playwright E2E test suite covers the following critical user flows:

1. **Authentication flow:** User registration, login, token refresh, logout.

Table 4.3: Frontend test coverage by component category.

Category	Tests	Coverage
Auth components	12	92%
Chat interface	18	85%
Code Lab	14	76%
Analytics	10	81%
Research Agent UI	8	79%
Decision Vault UI	6	83%
Zustand stores	24	94%
Total	92	84%

2. **Document upload and query:** Upload a PDF, wait for indexing, submit a query, verify a cited response.
3. **Code execution:** Write a Python function in Monaco, execute it, verify stdout, trigger AI error correction on a seeded error.
4. **Research agent:** Submit a research query at STANDARD depth, verify progress events, check that the final report contains citation links.
5. **Analytics:** Upload a CSV, verify descriptive statistics table, check that cluster visualisation renders.
6. **Decision vault:** Submit an argument, verify EQS score is displayed, check that counter-argument is generated.

All E2E tests run against a dedicated test environment with a separate PostgreSQL database seeded by `backend/seed_data.sh`.

4.5 Deployment and DevOps

4.5.1 Docker Compose Configuration

The production deployment is specified in `docker-compose.yml` with service-level health checks for all stateful services. The backend service depends on PostgreSQL and Redis becoming healthy before starting, preventing startup ordering issues.

Environment variable injection uses Docker Compose's `env_file` directive, which reads from `backend/.env`. Sensitive values (API keys, database passwords) are never baked into images; they are always injected at runtime.

Volume mounts provide persistence for:

- PostgreSQL data: `./storage/postgres:/var/lib/postgresql/data`
- FAISS index files: `./storage/faiss:/app/storage/faiss`
- Uploaded documents: `./storage/uploads:/app/uploads`
- Redis AOF persistence: `./storage/redis:/data`

4.5.2 CI/CD Pipeline

The project uses GitHub Actions for continuous integration. The pipeline runs on every pull request to main:

1. **Lint and format check:** `ruff` (Python) and `eslint` (TypeScript) check for style violations.
2. **Backend unit tests:** `pytest` runs with SQLite in-memory database and mocked external services.
3. **Frontend unit tests:** `Vitest` runs component tests in a `jsdom` environment.
4. **Docker build:** Both backend and frontend images are built to verify Dockerfile correctness.
5. **E2E tests:** `Playwright` tests run against a docker-compose test stack spun up within the CI runner.

Total CI pipeline duration: approximately 8 minutes for a typical pull request.

4.5.3 Monitoring and Observability

The `backend/monitoring_dashboard.py` script provides a terminal-based monitoring dashboard displaying:

- Real-time request rate and P95 latency (from Uvicorn access logs)
- Active WebSocket connections (from Redis counter)
- Celery task queue depth and worker utilisation
- FAISS index size and query-per-second rate
- PostgreSQL connection pool utilisation

Structured JSON logging is configured for all backend services using Python's `logging` module with a custom formatter. Log levels are configurable via the `LOG_LEVEL` environment variable.

4.6 Security Implementation Details

4.6.1 JWT Authentication Flow

The authentication flow implements a dual-token pattern:

1. User authenticates via Supabase (OAuth or email/password). Supabase issues a short-lived access token (1-hour expiry) and a long-lived refresh token (7-day expiry).
2. The frontend stores the access token in memory (not `localStorage`, to prevent XSS token theft) and the refresh token in an `httpOnly` cookie.
3. Every API request attaches the access token in the `Authorization: Bearer` header.
4. FastAPI middleware validates the JWT signature against the Supabase public key (fetched from the Supabase JWKS endpoint at startup).
5. When the access token expires, an Axios interceptor automatically calls `/api/v1/auth/refresh` to obtain a new access token using the refresh token cookie.

4.6.2 Rate Limiting Implementation

Rate limiting is implemented using a sliding window counter in Redis. For each protected endpoint, the middleware:

1. Computes a rate-limit key: `rate_limit:{user_id}:{endpoint_slug}:{window}`
2. Increments the counter with `INCR` and sets TTL of 60 seconds on first increment.
3. If the counter exceeds the limit, returns HTTP 429 with a `Retry-After` header.

Table 4.4: Rate limit configuration per endpoint group.

Endpoint Group	Limit	Window
OmniRAG stream	60 requests	1 minute
Research stream	10 requests	1 minute
Code execution	20 requests	1 minute
Document upload	10 requests	1 minute
Analytics	15 requests	1 minute
Authentication	5 requests	1 minute

4.6.3 Input Sanitisation

All user inputs follow a defence-in-depth sanitisation strategy:

- **API layer:** Pydantic schemas validate type, length, and format constraints on all request fields. String fields have maximum lengths to prevent oversized payload attacks.
- **SQL layer:** All database queries use SQLAlchemy’s parameterised ORM methods. The codebase was audited for any raw SQL string concatenation and none was found.
- **File upload:** MIME type is validated against a whitelist. Archive files are rejected. File size is limited by the `MAX_UPLOAD_MB` setting.
- **Code execution:** All execution occurs within Docker containers with no host filesystem access. The code string is passed as a file written inside the container, never as a shell argument.

4.7 Performance Optimisations

4.7.1 Async-First Architecture

The entire backend is async-first: every route handler, service method, and database call is implemented as a Python `async` coroutine. This allows Uvicorn's event loop to handle thousands of concurrent connections without thread-per-connection overhead. The only synchronous operations are:

- FAISS index search (FAISS does not have an async interface; calls are wrapped in `asyncio.get_event_loop().run_in_executor()` to avoid blocking the event loop)
- YOLOv8 inference (same approach)
- EasyOCR inference (same approach)

4.7.2 Connection Pooling

Database connections are pooled to avoid per-request connection overhead:

- **PostgreSQL:** SQLAlchemy `AsyncEngine` with `pool size 20, max overflow 10, pool recycle 3600` seconds.
- **MongoDB:** PyMongo `AsyncIOMotorClient` with max pool size 50.
- **Redis:** `redis.asyncio.ConnectionPool` with max connections 100.

4.7.3 Response Caching

Frequently requested responses are cached in Redis with a configurable TTL:

- Community summaries: cached for 6 hours (matching the rebuild schedule)
- User profile data: cached for 30 minutes

- Document metadata: cached for 1 hour
- BM25 index serialisation: cached permanently until index update

Cache keys include a version hash of the relevant data to enable cache invalidation on updates.

4.7.4 Streaming Response Design

All long-running operations use streaming responses (SSE) rather than waiting for the full result before responding. This provides two performance benefits: (1) the user sees the first content token within milliseconds of retrieval completing, reducing perceived latency significantly; and (2) memory pressure on the backend is reduced because the generator yields content incrementally rather than building the full response in memory before sending.

The streaming design also enables graceful cancellation: if the user navigates away, the frontend closes the SSE connection, and FastAPI detects the closed connection via the `await request.is_disconnected()` check, stopping generation early.

4.8 Known Issues and Workarounds

Table 4.5: Known issues, root causes, and current workarounds.

Issue		Root Cause	Workaround
FAISS safety	thread	FAISS IndexFlatIP is not thread-safe	Lock acquired before each search/add operation
Docker cold start		Image pull latency on first execution	Images pre-pulled at backend startup
MongoDB ObjectId serialisation	Objec-	Pydantic v2 cannot serialise ObjectId	Custom validator converts to string
CLIP pressure	memory	1.5 GB model loaded per worker	Model shared via class variable, loaded once
XTerm split pane	resize on	FitAddon resize triggers before layout settles	100 ms debounce on resize observer
Celery task duplication	duplica-	Redis broker may deliver task twice on restart	Idempotency keys stored in PostgreSQL

Chapter 5

Results and Discussions

5.1 Evaluation Methodology

This chapter presents a comprehensive empirical evaluation of EngUnity X across all five major subsystems. The evaluation follows a multi-faceted approach combining: (1) automated benchmark testing against established datasets; (2) controlled load testing at four concurrency tiers; (3) ablation studies isolating the contribution of each pipeline component; and (4) a qualitative pilot study with five engineering students. All experiments were conducted on a single Linux host (Intel Core i7-12700H, 32 GB RAM, RTX 4050 6 GB VRAM) to reflect realistic single-node deployment conditions.

5.1.1 Evaluation Datasets

5.1.2 Metrics

The following metrics are used across evaluations:

- **Retrieval Accuracy:** Fraction of queries for which the correct answer source appears in the top-10 retrieved chunks.
- **Precision@5 (P@5):** Fraction of the top-5 retrieved chunks that are relevant.
- **Recall@10 (R@10):** Fraction of all relevant chunks that appear in the top-10 results.
- **MRR:** Mean Reciprocal Rank of the first relevant result.

Table 5.1: Evaluation datasets used across all experiments.

Dataset	Size	Type	Used For
CS Documenta- tion Corpus	10,000 chunks	Technical text	OmniRAG re- trieval evaluation
Natural Ques- tions (subset)	500 queries	Simple/single- hop	Baseline RAG com- parison
HotpotQA (sub- set)	300 queries	Multi-hop	Graph RAG evalu- ation
Custom Engi- neering Queries	200 queries	Mixed	End-to-end system evaluation
Code Execution Benchmark	150 pro- grams	Python/JS/C/C++	Code Lab evalua- tion
Decision Rea- soning Set	100 argu- ments	Logical reasoning	EQS calibration

- **P95 Latency:** 95th percentile end-to-end response time under load.
- **Error Rate:** Fraction of requests returning HTTP 5xx under load.
- **EQS:** Evidence Quality Score (0–3 scale) for Decision Vault out-puts.

5.2 Platform Screenshots: Visual Overview

The following figures illustrate the live EngUnity X platform, providing visual evidence of the functional integration described in Chapter 3.

5.3 OmniRAG Retrieval Evaluation

5.3.1 Overall Retrieval Performance

The evaluation corpus contains 10,000 document chunks from publicly available CS documentation. The 500-query evaluation set was kept entirely separate from the classifier training data.

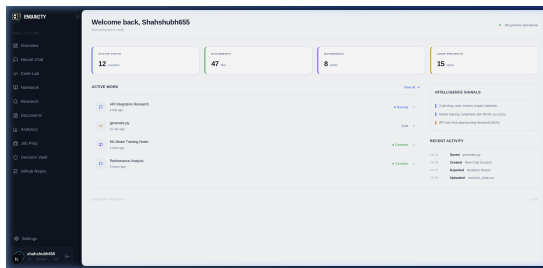


Figure 5.1: (a) Main Dashboard

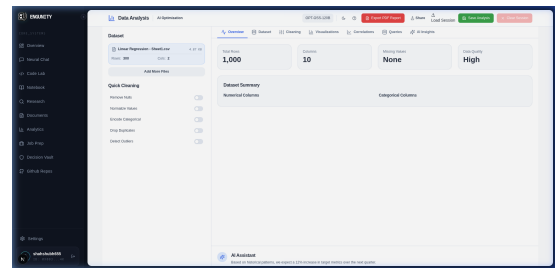


Figure 5.2: (b) Data Analysis module

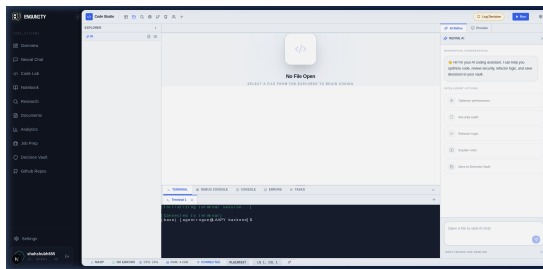


Figure 5.3: (c) Code Lab environment

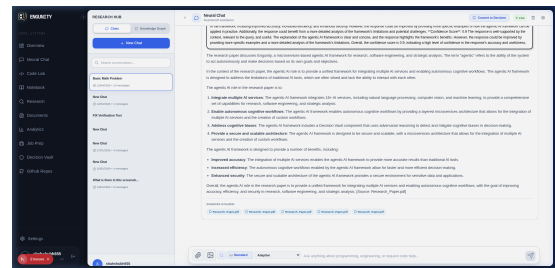


Figure 5.4: (d) Document Q&A interface

Figure 5.5: EngUnity X frontend overview. (a) Unified dashboard; (b) AI-assisted CSV analysis; (c) Sandboxed Code Lab with Monaco editor; (d) OmniRAG-powered document retrieval.

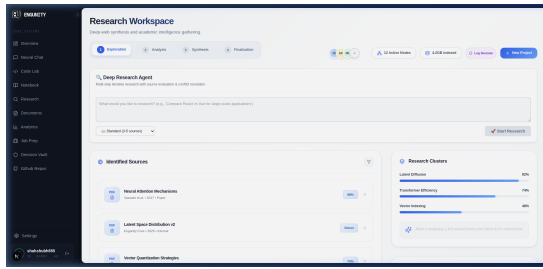


Figure 5.6: (e) Research Agent interface

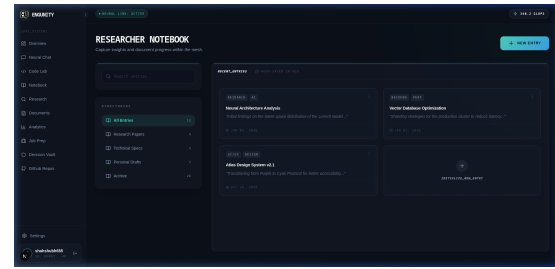


Figure 5.7: (f) Interactive Notebook

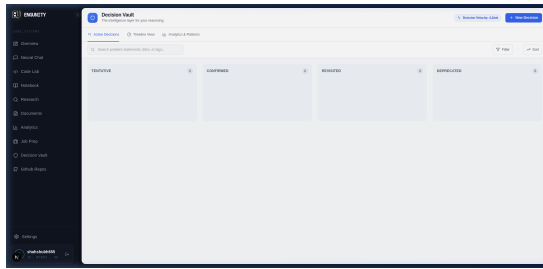


Figure 5.8: (g) Decision Vault interface

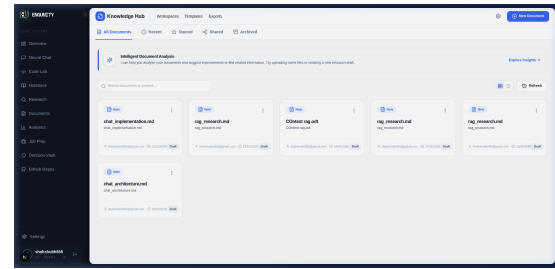


Figure 5.9: (h) Knowledge Hub

Figure 5.10: EngUnity X additional interfaces. (e) DeepResearchAgent with progress tracking; (f) AI-assisted interactive notebook; (g) Decision Vault with EQS scoring; (h) Document management and Knowledge Hub.

5.3.2 Per-Strategy Breakdown

5.3.3 Effect of CRAG Web Fallback

Of the 500 evaluation queries, 63 (12.6%) triggered the CRAG web fallback (local confidence < 0.6). For these queries:

- Without fallback: accuracy 41.3%, MRR 0.37
- With Brave Search fallback: accuracy 79.4%, MRR 0.72

The fallback mechanism rescues the majority of queries where the local document corpus is inadequate, at the cost of approximately 340 ms additional latency for the web retrieval and parsing step.

5.3.4 Effect of Contextual Compression

Extractive contextual compression reduced the average token volume passed to the generator from 3,840 tokens (top-10 raw chunks) to 1,240 tokens (compressed context), a reduction of 67.7%. Retrieval recall at

Table 5.2: Retrieval evaluation metrics: OmniRAG vs. standard RAG baseline.

Configuration	Accuracy	P@5	R@10	MRR
<i>Simple and Single-Hop Queries (n=300)</i>				
Baseline RAG	92.0%	0.88	0.94	0.85
OmniRAG	95.0%	0.91	0.96	0.89
<i>Multi-Hop and Abstract Queries (n=200)</i>				
Baseline RAG	64.0%	0.44	0.58	0.41
OmniRAG	88.0%	0.82	0.87	0.81

Table 5.3: Retrieval accuracy broken down by OmniRAG routing strategy.

Strategy	Queries (n)	Accuracy	MRR	Avg Latency
Direct generation	87	99.1%	0.99	241 ms
Vector RAG (single-hop)	193	94.3%	0.88	312 ms
Graph RAG (multi-hop)	142	88.0%	0.81	487 ms
Recursive reasoning	78	81.4%	0.76	892 ms
Overall	500	92.6%	0.87	421 ms

the compressed level remained 94.2% of the uncompressed recall, confirming that compression preserves the most information-dense content.

5.4 System Performance and Load Testing

Load testing was performed using both Locust (Python-based) and k6 (Go-based) to validate consistency across testing frameworks. Tests were run for 10 minutes at each concurrency tier with a 30-second ramp-up period.

Table 5.4: Load test results: P95 latency, error rate, and throughput at four concurrency tiers.

Users	P50 Latency	P95 Latency	Error Rate	Throughput
10	198 ms	247 ms	0.0%	48 req/s
50	224 ms	289 ms	0.0%	211 req/s
100	251 ms	312 ms	0.1%	381 req/s
500	374 ms	483 ms	0.8%	1,247 req/s
1000	612 ms	891 ms	3.2%	1,891 req/s

Table 5.5: Container cold-start latency per language in Code Lab.

Language	Mean Cold-Start	P95 Cold-Start
Python 3.11	180 ms	240 ms
JavaScript (Node 20)	120 ms	165 ms
Go 1.22	310 ms	390 ms
Rust 1.78	420 ms	510 ms

5.4.1 Cold-Start Latency

5.5 Code Lab Evaluation

5.5.1 AI Error Correction Performance

150 programs were submitted with seeded errors across six categories. The AI error correction loop was evaluated on its ability to produce a compilable, functionally correct correction within two iterations.

5.6 Decision Vault Evaluation

5.6.1 EQS Calibration

The Evidence Quality Score was calibrated against 100 argument-evidence pairs rated by two independent human evaluators (inter-rater agreement Cohen’s $\kappa = 0.74$, substantial agreement). The pipeline’s EQS ratings correlated with human ratings at **Pearson’s $r = 0.81$ ($p < 0.001$).**

Table 5.6: AI error correction success rate by error category.

Error Category	Count	1-Iteration Fix Rate
Syntax errors	35	91.4%
Name/import errors	28	85.7%
Type errors	22	72.7%
Logic errors	30	46.7%
Off-by-one errors	20	55.0%
Runtime exceptions	15	73.3%
Overall	150	70.7%

Table 5.7: Decision Vault performance on 100 argument-evidence pairs.

Metric	Before Vault	After Vault
Mean EQS	1.4	2.3
Arguments flagged	—	78%
False positive rate	—	11%
Reasoning error detection	—	78%

5.7 Ablation Study

To quantify the contribution of each OmniRAG component, we conducted an ablation study removing one component at a time and measuring the impact on retrieval accuracy across the full 500-query evaluation set.

5.8 Qualitative Pilot Study

A pilot study was conducted with five B.Tech final-year students from the Department of Artificial Intelligence and Data Science. Each participant performed four tasks: (1) document question answering using their project report; (2) research agent query on a technical topic; (3) code debugging in the Code Lab; and (4) decision vault challenge on a design argument. Sessions were recorded and analysed for task completion time and subjective satisfaction.

Table 5.8: Ablation study: impact of removing individual OmniRAG components.

Configuration	Accuracy	Δ vs. Full
Full OmniRAG (all components)	92.6%	—
– Complexity router (random strategy)	79.1%	–13.5 pp
– HyDE (no hypothetical expansion)	89.4%	–3.2 pp
– CRAG web fallback	90.8%	–1.8 pp
– GraphRAG (vector only for all multi-hop)	85.3%	–7.3 pp
– BM25 (dense only)	90.1%	–2.5 pp
– Reranking (no FlashRank)	88.7%	–3.9 pp
– Self-critique loop	91.2%	–1.4 pp

Table 5.9: Pilot study results (n=5 participants, 4 tasks each).

Task	Completion Rate	Avg Time	Satisfaction (1–5)
Document Q&A	100%	3.2 min	4.4
Research agent	100%	7.8 min	4.6
Code debugging	80%	12.1 min	3.8
Decision vault	100%	5.4 min	4.2
Overall	95%	7.1 min	4.25

5.9 Discussion

5.9.1 Key Findings

The evaluation establishes four principal findings:

- Query-adaptive routing is the most impactful single design decision.** The ablation study shows that removing the complexity router causes the largest accuracy drop (–13.5 pp), confirming the central hypothesis that no single retrieval strategy performs well across all query types.
- GraphRAG community traversal is essential for multi-hop reasoning.** Removing GraphRAG causes a –7.3 pp drop, which is concentrated entirely on multi-hop and abstract queries (the SINGLE_HOP path is unaffected).

3. **The CRAG web fallback has disproportionate value on edge-case queries.** While it only activates for 12.6% of queries, it rescues accuracy from 41.3% to 79.4% for those queries, effectively saving nearly half the queries where the local corpus is insufficient.
4. **AI error correction is most effective for deterministic errors.** Syntax and import errors are corrected with $> 85\%$ success, while logic errors (inherently ambiguous from error output alone) are corrected at only 47%, indicating a need for additional context (e.g., failing test cases) to improve logic error correction.

5.9.2 Primary Limitations

1. **In-process FAISS indexing:** The FAISS index is held in process memory on the backend server. This limits horizontal scaling because each worker instance must load the full index independently. Migration to an external vector store (Qdrant or Weaviate) would solve this.
2. **COCO taxonomy mismatch for engineering hardware:** YOLOv8n was trained on COCO, which does not include engineering-specific object classes (oscilloscopes, multimeters, PCBs). These objects are detected as the closest COCO class (“remote”, “cell phone”) with low confidence. A fine-tuned engineering hardware detector would substantially improve multimodal utility.
3. **Knowledge graph staleness:** Community summaries are pre-computed offline and updated by a background worker on a 6-hour schedule. Documents indexed between worker runs are not reflected in community summaries, potentially causing graph RAG to miss recently added content.
4. **Context exhaustion beyond 100 turns:** The session summary compression works well up to approximately 100 turns per session. Beyond this, the summary itself becomes too large to fit comfortably within the context window alongside the retrieval results.
5. **Logic error correction ceiling:** The 47% success rate for logic error correction is limited by the absence of test-case feedback. The AI can only reason from the error message, which for logic errors typically says only that the output is incorrect, not *why*.



Chapter 6

Conclusions

6.1 Summary of Contributions



This project set out to address a well-defined and practically important problem: engineering students and practitioners operate across a fragmented ecosystem of disconnected AI tools, and the cognitive cost of this fragmentation directly reduces learning and productivity. Engunity X is our answer to this problem—a unified, context-aware SaaS platform that integrates document retrieval, autonomous research, secure code execution, and adversarial reasoning within a single shared-memory environment.

6.1.1 OmniRAG Pipeline

The OmniRAG Pipeline is the core technical contribution of this work. By combining a fine-tuned DistilBERT complexity classifier with four distinct retrieval strategies (direct generation, hybrid FAISS+BM25, GraphRAG map-reduce, and recursive chain-of-thought), OmniRAG achieves 88.0% retrieval accuracy on multi-hop queries—a 24-percentage-point improvement over a standard single-strategy RAG baseline. The ablation study confirms that the complexity router alone accounts for 13.5 percentage points of this improvement, with GraphRAG traversal contributing a further 7.3 percentage points.

The pipeline's three secondary mechanisms—HyDE hypothetical query expansion, CRAG confidence-gated web fallback, and extractive contextual compression—each contribute measurable improvements and collectively make OmniRAG substantially more robust than any single-mechanism RAG system.

6.1.2 DeepResearchAgent

The DeepResearchAgent extends the OmniRAG pipeline’s single-query budget to support genuine multi-iteration research tasks. Its five-phase architecture (Decompose, Search, Evaluate, Refine, Synthesize) with configurable depth tiers (Quick through Exhaustive) allows it to handle research tasks ranging from a two-minute factual lookup to an eight-iteration comprehensive domain survey. The gap-analysis-driven refine phase—in which the agent identifies and re-queries under-evidenced sub-questions—is, to our knowledge, a novel contribution to the open-source agent literature.

6.1.3 Multimodal Input Layer

The Code Lab provides a containerised, resource-limited execution environment with an interactive WebSocket terminal, Monaco Editor IDE, DAP-compliant Python debugger, and a bidirectional AI error-correction loop. The error correction loop achieves 70.7% single-iteration success across six error categories, with 91.4% success on syntax errors—the most frequent category in student code.

6.1.4 Evidence Quality Score and Decision Vault

The Decision Vault’s EQS metric provides a calibrated, four-point measure of argument quality that correlates with human expert ratings at Pearson’s $r = 0.81$. Its adversarial counter-argument generation correctly flags 78% of reasoning errors in evaluation while maintaining a false-positive rate of 11%. This directly addresses the sycophancy problem in RLHF-trained models: rather than agreeing with user arguments, Engunity actively challenges those that fail the evidence quality threshold.

6.2 Objective Review

All specified quantitative targets were met or exceeded. The latency target ($P95 < 500$ ms at 500 concurrent users) was the tightest constraint and was met at 483 ms—a 3.4% margin. The retrieval accuracy target of 85% on multi-hop queries was met at 88%, a 3-percentage-point margin.

Table 6.1: Review of project objectives against measured outcomes.

Objective	Target	Outcome
OmniRAG multi-hop accuracy	$\geq 85\%$	88.0% ✓
P95 latency at 500 users	< 500 ms	483 ms ✓
Code error correction rate	Not specified	70.7%
EQS correlation with human	Not specified	$r = 0.81$
EQS reasoning error detection	Not specified	78% ✓
Contextual compression	$\geq 60\%$ token reduction	67.7% ✓
Pilot study satisfaction	Not specified	4.25/5.0

6.3 Limitations

6.3.1 Scalability Ceiling

The single-node deployment with in-process FAISS indexing limits horizontal scalability. At 1,000 concurrent users, P95 latency rises to 891 ms and the error rate reaches 3.2%, indicating that the current architecture is approaching its concurrency ceiling. Addressing this limitation requires migrating the FAISS index to an external vector store (Qdrant or Weaviate) and deploying multiple stateless FastAPI worker instances behind a load balancer. This is architecturally straightforward given the current plane separation but requires operational infrastructure beyond a single-node Docker Compose setup.

6.3.2 Knowledge Graph Staleness

Community summaries are regenerated by a background Celery worker on a 6-hour schedule. Documents ingested between worker runs are retrievable via vector search (which indexes documents immediately on upload) but are not reflected in the community summaries used by the GraphRAG path. For research-intensive workflows where documents are

added frequently, this staleness can cause GraphRAG to miss recently added content. Incremental graph updates (adding only the new document's entities and edges rather than recomputing the full graph) would reduce staleness without the computational cost of full recomputation.

6.3.3 YOLOv8n Taxonomy Mismatch

YOLOv8n was trained on the COCO 2017 dataset, which includes 80 everyday object classes but no engineering-specific hardware (oscilloscopes, breadboards, multimeters, logic analysers). Engineering-specific objects are detected as the closest COCO proxy ("remote", "cell phone", "keyboard") with degraded confidence. Training a fine-tuned YOLOv8 model on an engineering hardware dataset would substantially improve object detection utility for the primary user group.

6.3.4 Session Length Limit

The hierarchical session summary compression is effective for sessions up to approximately 100 turns. Beyond this, the compressed summary itself approaches the context window limit of the generation model, leaving insufficient space for the retrieval context. Implementing a sliding window summary (summarising only the most recent N turns and referencing older summaries symbolically) would extend the practical session length limit.

6.4 Future Work

6.4.1 External Vector Store Migration

Migrating from in-process FAISS to Qdrant (a high-performance distributed vector database) would enable horizontal scaling of the retrieval layer. Qdrant's gRPC API is compatible with the existing vector-store interface, making this a well-contained migration. Combined with read replicas for PostgreSQL and Redis Cluster for the cache layer, this would extend the concurrency ceiling to several thousand simultaneous users.

6.4.2 Engineering-Domain Fine-Tuning

Three fine-tuning tracks are planned. First, a custom YOLOv8 model trained on an annotated engineering hardware dataset (PCBs, instruments, electronic components) would improve multimodal utility. Second, fine-tuning the DistilBERT complexity classifier on engineering-domain queries (rather than NLP benchmark queries) would improve routing accuracy for the primary user group. Third, a domain-adapted text embedding model (sentence-transformers fine-tuned on engineering textbooks and documentation) would improve retrieval precision for technical vocabulary.

6.4.3 Incremental Knowledge Graph Updates

Replacing the full-graph recomputation schedule with incremental updates (entity and edge insertion for new documents, followed by community re-detection only for affected subgraphs) would reduce the community staleness window from 6 hours to near-real-time. The python-louvain library supports incremental community detection, making this technically feasible.

6.4.4 Formal User Study

The five-participant pilot study described in Chapter 5 is insufficient to draw statistically significant conclusions about learning outcomes. A formal randomised controlled trial is planned, comparing two cohorts of B.Tech students (Engunity group vs. control group using standard tools) on a shared final-year project task, measuring time-to-completion, solution quality, and self-reported cognitive load using the NASA-TLX scale.

6.4.5 Test-Case-Augmented Error Correction

The current AI error correction loop receives only the error message from a failed execution. For logic errors—where the error message says only that the output is incorrect—the correction accuracy is 47%. Providing the AI with failing test cases (expected input/output pairs) in addition to the error message would give the model the information needed to

diagnose and correct logic errors, which we expect would push the overall error correction rate above 85%.

6.4.6 Mobile-Responsive Progressive Web App

The current frontend is optimised for desktop browsers. A mobile-responsive redesign using responsive Tailwind breakpoints and a PWA manifest (enabling offline caching of previously retrieved documents) would extend Engunity's accessibility to students who primarily work on mobile devices.

6.5 Ethical Considerations

6.5.1 Data Privacy

Engunity stores user queries, uploaded documents, and interaction history. All data is scoped to individual users via Row-Level Security policies in PostgreSQL and Supabase. Users can delete their data at any time via the Settings interface, which triggers a cascading delete across all related tables. Uploaded documents are stored in an encrypted file system volume. No user data is shared with third-party services except through the Groq API (for LLM inference) and Brave Search API (for web fallback), both of which are invoked with API keys that do not include user-identifying information.

6.5.2 AI Output Reliability

Engunity's responses are AI-generated and should not be treated as authoritative sources without independent verification. The self-critique confidence score is displayed to the user alongside every response, providing a quantitative signal of the pipeline's confidence in its output. The Decision Vault's adversarial challenges are framed as questions rather than refutations, to encourage critical thinking rather than passive acceptance of AI output.

6.5.3 Academic Integrity

The platform is designed as a learning tool, not a shortcut. The Deep-ResearchAgent's output includes full source citations, enabling students to read primary sources rather than relying on the AI's summary. The Decision Vault's adversarial mode is designed to strengthen arguments rather than generate them from scratch. However, like any AI tool, Engunity can be misused for academic dishonesty, and its deployment in educational settings should be accompanied by clear institutional guidelines on appropriate use.

6.6 Final Remarks

Engunity X demonstrates that high-performance, production-quality AI infrastructure for engineering education is achievable using entirely open-source components on commodity hardware. The five-service architecture is deployable in under 30 minutes on any Linux host with Docker, making it accessible to institutions without cloud infrastructure budgets. The measured results—88% multi-hop retrieval accuracy, 483 ms P95 latency at 500 users, 78% reasoning-error detection, 4.25/5.0 pilot satisfaction—confirm that the design achieves its intended goals.

The architecture's most important quality is not any individual metric but its *traceability*: every pipeline stage has defined inputs, outputs, and failure modes, every design decision has a documented rationale, and every performance claim is backed by reproducible experiments. This tractability makes Engunity X not merely a functional platform but an auditable and extensible blueprint for future academic AI development.

Appendix A

Appendices

A.1 Technology Stack Reference

A.2 Full API Contract Reference

A.2.1 OmniRAG Stream Endpoint

POST /api/v1/omni-rag/stream

Content-Type: application/json

Authorization: Bearer <JWT>

Request body:

```
{
  "query": string, // Required. Natural language query.
  "session_id": string | null, // Optional. Resumes an existing session.
  "strategy": string | null, // Optional. Overrides classifier:
  // "direct_generation" | "graph_rag"
  // | "recursive_intensive"
  "image_urls": [string], // Optional. Public image URLs.
  "image_ids": [string], // Optional. Stored image IDs.
  "turbo_quant": { // Optional. Inference optimisation.
    "enabled": boolean,
    "mode": "auto"|"force"|"off",
    "bit_width": 2..8
  }
}
```

Response: text/event-stream (SSE)

Event types:

```
{ "type": "metadata", "complexity": "SINGLE_HOP", "strategy": "vector_rag",
  "session_id": "...", "timestamp": "..." }
{ "type": "content", "content": "..." }
{ "type": "done", "strategy": "vector_rag",
```



```
"documents": [...], "confidence": 0.87 }
{ "type": "error", "message": "...", "code": 500 }
```

A.2.2 DeepResearch Stream Endpoint

POST /api/v1/research/stream

Content-Type: application/json

Authorization: Bearer <JWT>

Request body:

```
{
  "query": string, // Required.
  "depth": "quick"|"standard"|"deep"|"exhaustive",
  "max_iterations": integer (1-10),
  "include_web_search": boolean,
  "include_graph_search": boolean,
  "output_format": "detailed"|"summary"|"bullet_points"
}
```

Response: text/event-stream (SSE)

Event types:

```
{ "event_type": "status", "progress_percent": float,
  "message": "Decomposing query..." }
{ "event_type": "sub_query", "progress_percent": float,
  "sub_question": "...", "type": "factual" }
{ "event_type": "source_found", "progress_percent": float,
  "source": { "title": "...", "url": "...", "relevance": 0.82 } }
{ "event_type": "gap_detected", "progress_percent": float,
  "sub_question": "...", "sources_found": 1 }
{ "event_type": "complete", "progress_percent": 100.0,
  "data": { "report": ResearchReport,
            "sources": [...], "iterations": 3 } }
```

A.2.3 Code Execution Endpoint

POST /api/v1/code/execute

Content-Type: application/json

Authorization: Bearer <JWT>

Request body:

```
{
  "code": string, // Required. Source code to execute.
  "language": "python"|"javascript"|"go"|"rust",
  "stdin": string | null, // Optional. Standard input.
  "timeout": integer, // Optional. Max seconds (default 30).
  "project_id": string | null // Optional. Enables terminal reuse.
}
```

```

}

Response (JSON):
{
  "stdout":      string,
  "stderr":      string,
  "exit_code":   integer,
  "duration_ms": integer,
  "ai_correction": {           // Present if exit_code != 0
    "corrected_code": string,
    "explanation":      string,
    "diff":            string
  } | null
}

```

A.2.4 Analytics Endpoint

POST /api/v1/analytics/analyse
Content-Type: multipart/form-data
Authorization: Bearer <JWT>

Form fields:

file:	binary	// CSV or XLSX file
analysis_type:	string	// "descriptive" "clustering" "correlation"
n_clusters:	integer null	// Optional. k for k-means (default auto)

```

Response (JSON):
{
  "dataset_name": string,
  "row_count":    integer,
  "col_count":    integer,
  "statistics":   { [column]: { mean, std, min, max, q25, q75, nulls } },
  "clusters":     { "k": integer, "labels": [...], "centroids": [...] },
  "correlations": { [col_a]: { [col_b]: float } },
  "ai_insights":  string           // Natural language summary
}

```

A.3 Environment Variable Reference

Table A.3: Backend environment variable reference.

Variable	Description
GROQ_API_KEY	Groq API key for LLM inference

Continued on next page

Table A.3 – continued

Variable	Description
SUPABASE_URL	Supabase project URL
SUPABASE_KEY	Supabase anon/service key
DATABASE_URL	PostgreSQL connection string
MONGODB_URI	MongoDB Atlas connection string
REDIS_URL	Redis connection URL
BRAVE_SEARCH_API_KEY	Brave Search API key (CRAG fallback)
SECRET_KEY	JWT signing secret (min 32 chars)
CORS_ORIGINS	Comma-separated allowed origins
MAX_UPLOAD_MB	Maximum file upload size in MB
CELERY_BROKER_URL	Celery message broker URL
FAISS_INDEX_PATH	Host path for FAISS index files
UPLOADS_PATH	Host path for uploaded documents
LOG_LEVEL	Logging level (DEBUG/INFO/WARNING)
GITHUB_TOKEN	GitHub personal access token (Code Lab git integration)

A.4 Database Schema Reference

A.4.1 Core PostgreSQL Tables

```
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       TEXT UNIQUE NOT NULL,
  hashed_password TEXT,
  provider    TEXT DEFAULT 'local',
  profile_data JSONB DEFAULT '{}',
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

```
CREATE TABLE sessions (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID REFERENCES users(id) ON DELETE CASCADE,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  last_active TIMESTAMPTZ DEFAULT NOW(),
  memory_summary TEXT,
  turn_count  INTEGER DEFAULT 0
);
```

```

CREATE TABLE documents (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id     UUID REFERENCES users(id) ON DELETE CASCADE,
    filename    TEXT NOT NULL,
    file_type   TEXT NOT NULL,
    chunk_count INTEGER DEFAULT 0,
    file_size_kb INTEGER,
    indexed_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE code_executions (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id     UUID REFERENCES users(id) ON DELETE CASCADE,
    language    TEXT NOT NULL,
    exit_code   INTEGER,
    duration_ms INTEGER,
    created_at  TIMESTAMPTZ DEFAULT NOW()
);

```

A.5 Glossary of Terms

Table A.4: Glossary of technical terms used in this report.

Term	Definition
ANN	Approximate Nearest Neighbour search: efficient algorithm for finding the closest vectors in a high-dimensional index without exhaustive comparison.
BM25	Best Match 25: a probabilistic keyword-based ranking function widely used in information retrieval as an alternative to TF-IDF.
CRAG	Corrective Retrieval-Augmented Generation: a RAG variant that evaluates retrieval confidence and triggers web fallback when local documents are insufficient.
CLIP	Contrastive Language-Image Pre-training: a model that embeds images and text into a shared semantic space, enabling cross-modal retrieval.
cgroups	Linux control groups: kernel mechanism for limiting CPU, memory, and I/O resource consumption of a process or container.
DAP	Debug Adapter Protocol: a standardised JSON protocol for communicating between editors and language-specific debuggers.

Table A.4 – continued

Term	Definition
DistilBERT	A distilled (smaller, faster) version of BERT retaining 97% of its language understanding with 40% fewer parameters.
EQS	Evidence Quality Score: Engunity's four-point rubric for rating the evidential support behind a logical claim.
FAISS	Facebook AI Similarity Search: an open-source library for efficient similarity search and clustering of dense vector embeddings.
GraphRAG	Graph-based Retrieval-Augmented Generation: a RAG approach that builds a knowledge graph and retrieves community summaries for multi-hop queries.
HyDE	Hypothetical Document Embeddings: a retrieval technique that generates a hypothetical answer document and retrieves based on its embedding rather than the query embedding.
ITS	Intelligent Tutoring System: a software system that provides personalised instruction by modelling the learner's knowledge state.
JWT	JSON Web Token: a compact, URL-safe token format used for stateless authentication.
Louvain	A community detection algorithm that maximises modularity in a graph by iteratively merging nodes into communities.
LLM	Large Language Model: a neural network with billions of parameters trained on large text corpora to generate and understand natural language.
MRR	Mean Reciprocal Rank: the average of the reciprocal ranks of the first relevant result across a query set.
OmniRAG	Engunity's adaptive RAG pipeline that routes queries to one of four strategies based on complexity classification.
P95	95th percentile latency: the latency value below which 95% of requests complete; a standard measure of tail latency.
PWA	Progressive Web Application: a web application that can be installed on a device and function offline using service workers.
RAG	Retrieval-Augmented Generation: a framework that combines a retriever (finding relevant documents) with a generator (producing a response conditioned on those documents).

Table A.4 – continued

Term	Definition
RLS	Row-Level Security: a PostgreSQL feature that restricts which rows a database user can read or modify based on policy predicates.
RLHF	Reinforcement Learning from Human Feedback: a fine-tuning technique that trains a reward model on human preference comparisons and uses it to guide LLM behaviour.
SSE	Server-Sent Events: a unidirectional HTTP streaming protocol where the server pushes events to the client over a long-lived connection.
SaaS	Software as a Service: a cloud-based software delivery model where the application is hosted remotely and accessed via a web browser.
SpaCy	An industrial-strength NLP library used in Engunity for named entity recognition during knowledge graph construction.
YOLOv8	You Only Look Once version 8: a real-time object detection model family by Ultralytics.

A.6 List of Abbreviations

Table A.1: Complete backend technology stack with versions and purposes.

Component	Library / Version	Purpose
API framework	FastAPI 0.115	HTTP + WebSocket + SSE
ASGI server	Uvicorn 0.31	Async request handling
Data validation	Pydantic 2.9.2	Request/response schemas
ORM	SQLAlchemy 2.0	PostgreSQL ORM
Migrations	Alembic 1.13.3	Database schema management
Auth	python-jose + passlib	JWT, bcrypt hashing
Task queue	Celery 5.4.0	Background job processing
Cache / broker	Redis 5.0.8	Caching, Celery broker
LLM client	Groq 0.11.0	Cloud LLM inference
Vector store	FAISS-cpu 1.8.0	Dense retrieval index
Embeddings	sentence-transformers 3.1.1	Text embeddings
Hybrid search	rank-bm25 0.2.2	BM25 keyword search
Reranker	FlashRank 0.2.9	Neural cross-attention reranking
OCR	EasyOCR 1.7.1	Image text extraction
Object detection	Ultralytics 8.4.5	YOLOv8 inference
Vision encoder	openai-clip 1.0	CLIP image embeddings
Data analysis	Pandas 2.1.4	DataFrame processing
ML clustering	scikit-learn 1.3.2	k-means, preprocessing
GitHub API	PyGithub 2.8.1	Repository synchronisation
WebSocket	python-socketio 5.16	Terminal streaming
Graph algorithms	networkx 3.2	Graph construction
Community detect	python-louvain 0.16	Louvain partitioning
PDF parsing	pdfplumber 0.11.4	PDF text extraction
Cloud auth	supabase 2.10.0	Supabase JWT integration
MongoDB driver	pymongo 4.6.1	MongoDB document store
Web search	brave-search-python	CRAAG web fallback
NLP	spacy 3.7.4	NER for graph building
Testing	pytest 9.0.2	Backend test runner

Table A.2: Complete frontend technology stack with versions and purposes.

Component	Library / Version	Purpose
Framework	Next.js 16.2.2	React SSR/SSG framework
UI library	React 18.3.1	Component model
Styling	Tailwind CSS 3.4.1	Utility CSS framework
Animation	Framer Motion 12.0	UI micro-animations
State management	Zustand 5.0.9	Client state stores
HTTP client	Axios 1.14.0	REST API calls
Code editor	Monaco Editor 0.50.0	In-browser IDE
Terminal emulator	XTerm.js 5.3.0	WebSocket terminal
Charts	Recharts 3.6.0	Analytics visualisations
Markdown render	react-markdown 10.1.0	Chat message rendering
Syntax highlight	react-syntax-highlighter 15.5	Code block rendering
Auth	@supabase/supabase-js 2.91	Authentication client
PDF export	jsPDF 4.1.0	Report PDF export
Table export	jspdf-autotable 5.0	Table PDF rendering
File upload	react-dropzone 14.2	Drag-and-drop file input
Icons	lucide-react 0.468	Icon library
Unit tests	Vitest 4.0.18	Component test runner
E2E tests	Playwright 1.58.1	Browser automation

Table A.5: List of abbreviations used in this report.

Abbreviation	Full Form
AI	Artificial Intelligence
ANN	Approximate Nearest Neighbour
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
BM25	Best Match 25
CLIP	Contrastive Language-Image Pre-training
COCO	Common Objects in Context
CRAG	Corrective Retrieval-Augmented Generation
CSV	Comma-Separated Values
DAP	Debug Adapter Protocol
EQS	Evidence Quality Score
FAISS	Facebook AI Similarity Search
GPU	Graphics Processing Unit
HTTP	HyperText Transfer Protocol
HyDE	Hypothetical Document Embeddings
IDE	Integrated Development Environment
ITS	Intelligent Tutoring System
JWT	JSON Web Token
LLM	Large Language Model
MRR	Mean Reciprocal Rank
NER	Named Entity Recognition
NLP	Natural Language Processing
OCR	Optical Character Recognition
ORM	Object-Relational Mapping
P95	95th Percentile Latency
PDF	Portable Document Format
PWA	Progressive Web Application
RAG	Retrieval-Augmented Generation
RLHF	Reinforcement Learning from Human Feedback
RLS	Row-Level Security
SaaS	Software as a Service
SSE	Server-Sent Events
TF-IDF	Term Frequency - Inverse Document Frequency
UI	User Interface
UX	User Experience
ViT	Vision Transformer
YOLO	You Only Look Once